Time Series & Recurrent Neural Networks

ACTL3143 & ACTL5111 Deep Learning for Actuaries

Patrick Laub

UNSW SYDNEY

Lecture Outline

- **Introduction**
- Time series data
- Australian financial stocks
- Sydney weather
- Naive baseline forecasts
- Use the recent history
- Simple Recurrent Neural Networks
- Recurrent Neural Networks
- Multi-step forecasting and multivariate forecasting
- Car Crash NLP with RNNs

A lecture on sequence modelling

This is really a lecture about *sequences* — ordered data where the *order itself carries meaning*.

- *Time series*: stock prices, claim counts, mortality rates, temperatures.
- *Text*: a sentence is a sequence of words; a document a sequence of sentences.

The common thread: to predict the next element we must use what came *before*.

Lecture Outline

- Introduction
- **Time series data**
- Australian financial stocks
- Sydney weather
- Naive baseline forecasts
- Use the recent history
- Simple Recurrent Neural Networks
- Recurrent Neural Networks
- Multi-step forecasting and multivariate forecasting
- Car Crash NLP with RNNs

Tabular data vs time series data

Tabular data

We have a dataset $\{\boldsymbol{x}_i, y_i\}_{i=1}^n$ which we assume are i.i.d. observations.

Brand	Mileage	# Claims
BMW	101 km	1
Audi	432 km	0
Volvo	3 km	5
⋮	⋮	⋮

The goal is to *predict* the y for some covariates $\boldsymbol{x}$.

Time series data

Have a sequence $\{\boldsymbol{x}_t, y_t\}_{t=1}^T$ of observations taken at regular time intervals.

Date	Humidity	Temp.
Jan 1	60%	20 °C
Jan 2	65%	22 °C
Jan 3	70%	21 °C
⋮	⋮	⋮

The task is to *forecast* future values based on the past.

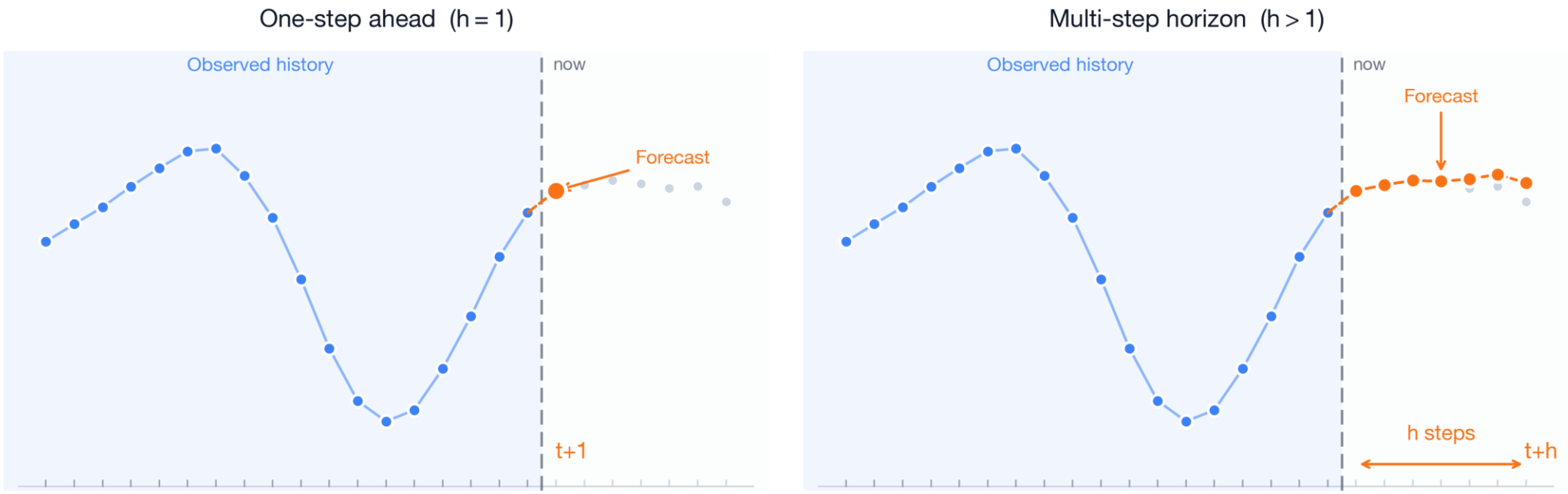
Attributes of time series data

- **Temporal ordering**: The order of the observations matters.
- **Trend**: The general direction of the data.
- **Noise**: Random fluctuations in the data.
- **Seasonality**: Patterns that repeat at regular intervals.

> **Question**
>
> What will be the temperature in Berlin tomorrow? What information would you use to make a prediction?

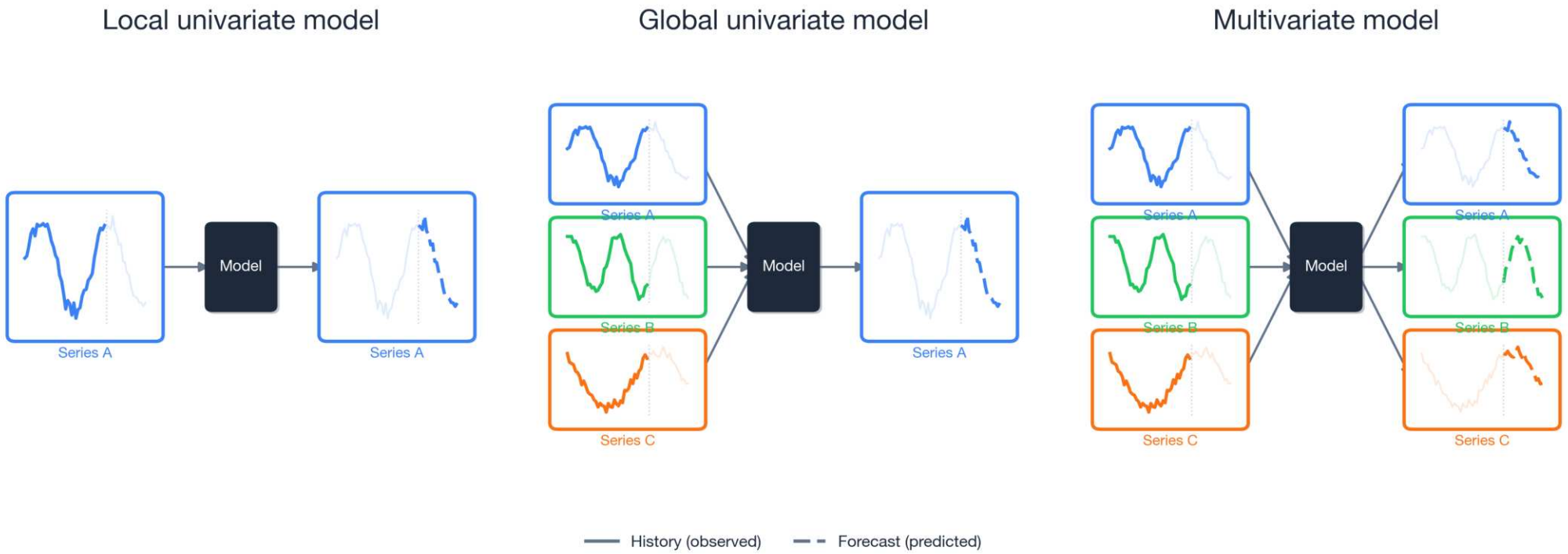
The forecasting problem

We stand at a *reference time* t: we have seen the series up to now, and want the next h values.



We need to know all covariates at time t to make a prediction at $t + 1$.

Three kinds of forecasting models



A schematic contrasting local univariate models (one series predicts itself), global univariate models (multiple series predict one), or multivariate models (multiple inputs, multiple targets).

For local models, you would have a separate model for each time series. Global models allow for *cross-learning*, and are more promising for deep forecasting techniques.

See Benidis et al. (2022, p. 4).

Time series for actuaries

Applications:

- *Claims reserving* — project future claim payments given a notified claim.
- *Mortality forecasting* — extrapolate mortality rates from life tables, e.g., Richman & Wuthrich (2019).
- *Economic scenario generation* — simulate interest rates, inflation, and asset returns.

Neural networks or traditional models:

- *Traditional* models (ARIMA, Lee–Carter) can be hard to beat on a *single, short, well-structured* series — decades of refinement, few parameters, robust.
- *Neural networks* work best with global cross-learning, and rich covariates.

Think about the fundamental predictability of the data:

- *Share prices* are close to a *random walk*, hopeless to predict.
- *Weather* is often genuinely predictable, especially in the short-term.

Lecture Outline

- Introduction
- Time series data
- **Australian financial stocks**
- Sydney weather
- Naive baseline forecasts
- Use the recent history
- Simple Recurrent Neural Networks
- Recurrent Neural Networks
- Multi-step forecasting and multivariate forecasting
- Car Crash NLP with RNNs

Imports needed for these demos

```
import re
import random
from pathlib import Path

import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import numpy as np
import pandas as pd

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer

import keras
from keras.models import Sequential
from keras.layers import (Dense, Input, Rescaling, Reshape,
    SimpleRNN, GRU, LSTM, Bidirectional, Embedding)
from keras.callbacks import EarlyStopping
```

Australian financial stocks

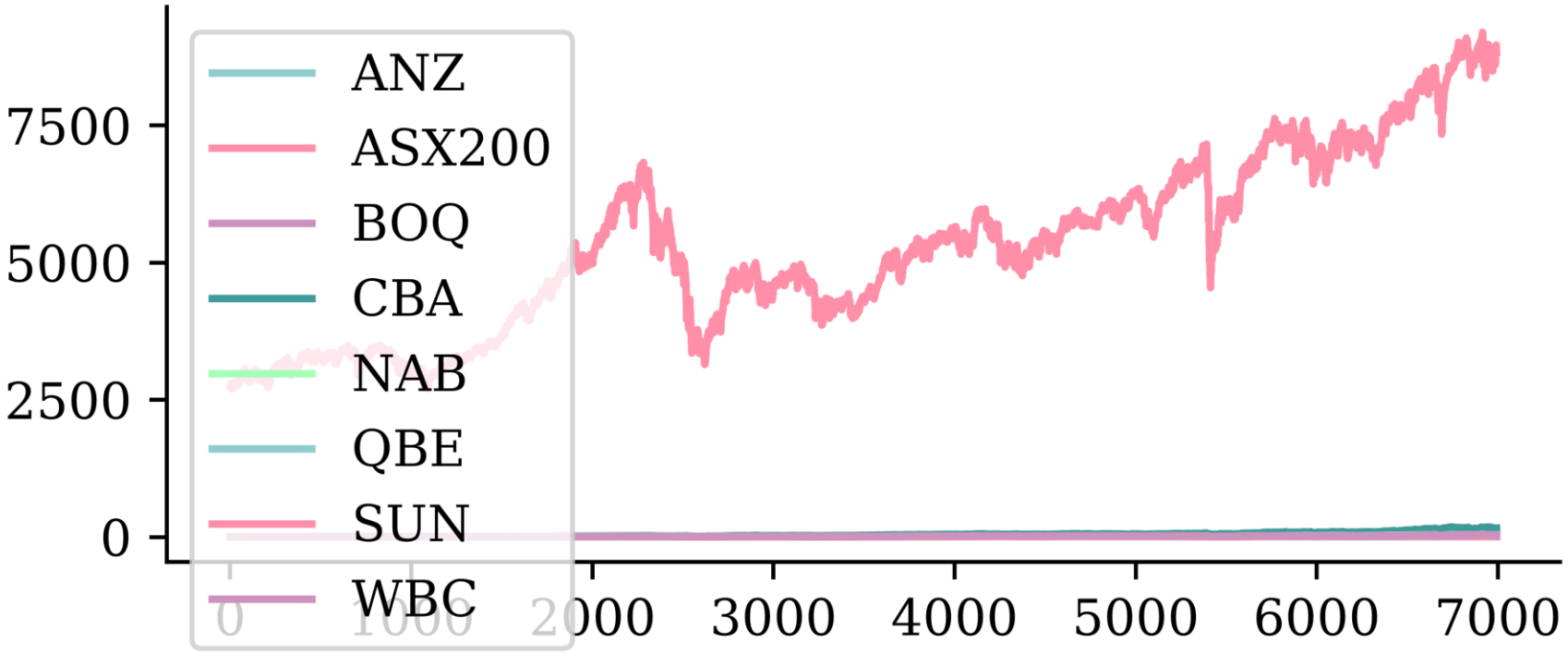
```
stocks = pd.read_csv("data/interim/aus_fin_stocks.csv")
stocks
```

	Date	ANZ	ASX200	BOQ	CBA	NAB	QBE	SUN	WBC
0	1999-01-01	2.327790	NaN	NaN	5.823670	NaN	NaN	1.894796	NaN
1	1999-01-04	2.345227	2732.199951	NaN	5.760781	5.195878	2.277954	1.894796	2.517086
2	1999-01-05	2.338688	2716.600098	NaN	5.763297	5.265624	2.205534	1.894796	2.562639
...	...	...	...	...	...	...	...	...	...
6990	2026-06-17	35.049999	8966.299805	6.37	163.710007	37.669998	23.570000	18.629999	35.560001
6991	2026-06-18	35.139999	8911.099609	6.30	162.229996	37.340000	24.010000	18.670000	35.160000
6992	2026-06-19	35.029999	8828.700195	6.32	162.399994	37.740002	24.059999	18.620001	35.009998

6993 rows × 9 columns

Plot

```
1 stocks.plot()
```



Question

What is wrong with this plot?

Data types and NA values

```
stocks.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6993 entries, 0 to 6992
Data columns (total 9 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    6993 non-null   str
 1   ANZ     6993 non-null   float64
 2   ASX200  6939 non-null   float64
 3   BOQ     6847 non-null   float64
 4   CBA     6993 non-null   float64
 5   NAB     6992 non-null   float64
 6   QBE     6992 non-null   float64
 7   SUN     6993 non-null   float64
 8   WBC     6992 non-null   float64
dtypes: float64(8), str(1)
memory usage: 560.1 KB
```

```
for col in stocks.columns:
    print(f"{col}: {stocks[col].isna().su
```

```
Date: 0
ANZ: 0
ASX200: 54
BOQ: 146
CBA: 0
NAB: 1
QBE: 1
SUN: 0
WBC: 1
```

```
asx200 = stocks.pop("ASX200")
```

UNSW SYDNEY

Set the index to the date

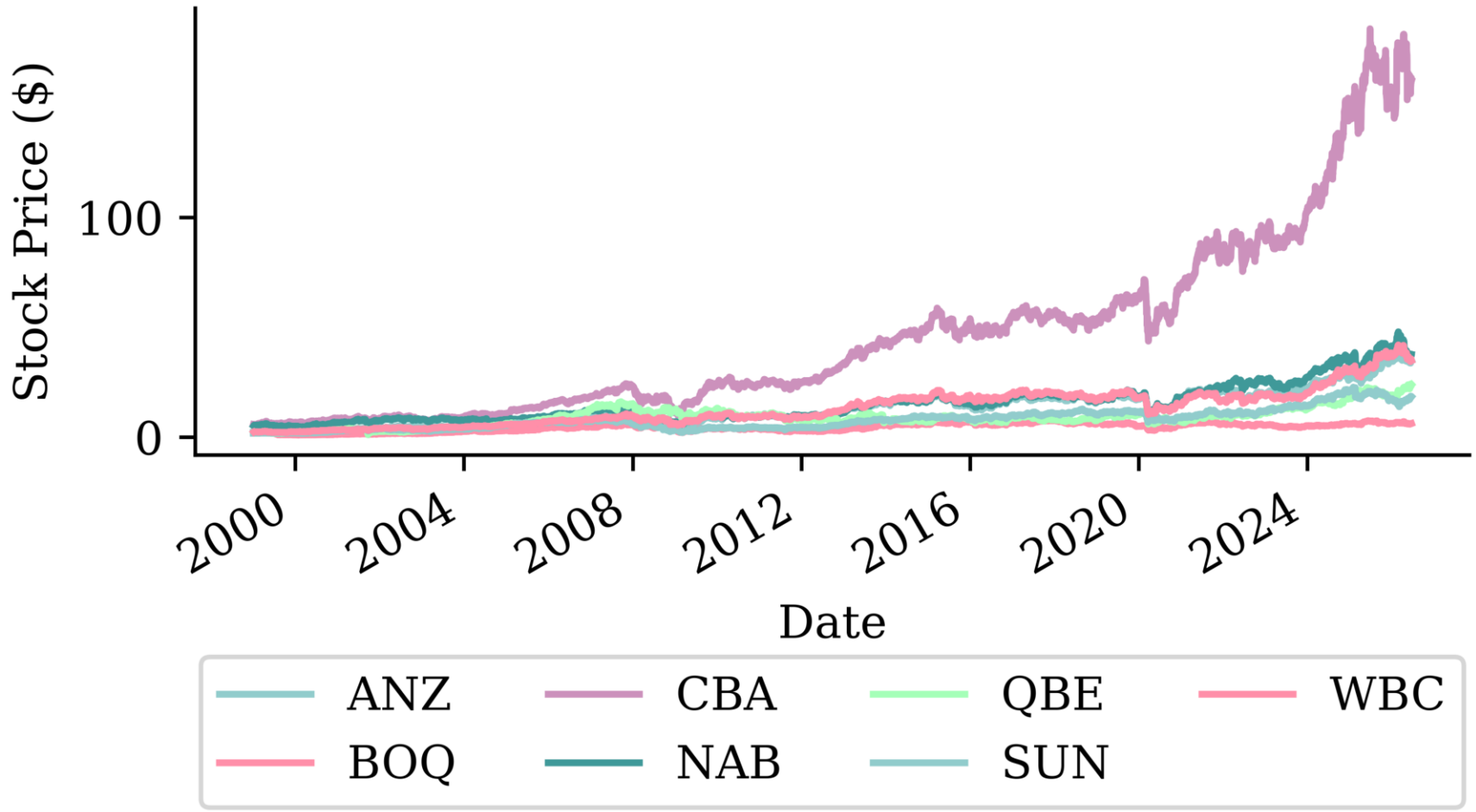
```
stocks["Date"] = pd.to_datetime(stocks["Date"])
stocks = stocks.set_index("Date")
stocks
```

	ANZ	BOQ	CBA	NAB	QBE	SUN	WBC
Date							
1999-01-01	2.327790	NaN	5.823670	NaN	NaN	1.894796	NaN
1999-01-04	2.345227	NaN	5.760781	5.195878	2.277954	1.894796	2.517086
1999-01-05	2.338688	NaN	5.763297	5.265624	2.205534	1.894796	2.562639
...	...	...	...	...	...	...	...
2026-06-17	35.049999	6.37	163.710007	37.669998	23.570000	18.629999	35.560001
2026-06-18	35.139999	6.30	162.229996	37.340000	24.010000	18.670000	35.160000
2026-06-19	35.029999	6.32	162.399994	37.740002	24.059999	18.620001	35.009998

6993 rows × 7 columns

Plot II

```
1 stocks.plot() # This plot can still be improved.. why?
2 plt.ylabel("Stock Price ($)")
3 plt.legend(loc="upper center", bbox_to_anchor=(0.5, -0.4), ncol=4);
```



Can index using dates I

```
1 stocks.loc["2010-1-4":"2010-01-8"]
```

Date	ANZ	BOQ	CBA	NAB	QBE	SUN	WBC
2010-01-04	9.092883	4.196682	24.627254	9.934697	13.140049	4.317703	9.993726
2010-01-05	9.136576	4.250533	24.999773	10.061602	13.207809	4.382517	10.076675
2010-01-06	9.001514	4.228993	25.125454	9.865807	13.004534	4.287787	10.029274
2010-01-07	8.787006	4.099753	24.883083	9.786038	12.769980	4.347618	9.894972
2010-01-08	8.838641	4.200274	25.206242	9.753405	12.910715	4.372547	9.934475

Note, these ranges are *inclusive*, not like Python's normal slicing.

Can index using dates II

So to get 2019's December and all of 2020 for CBA:

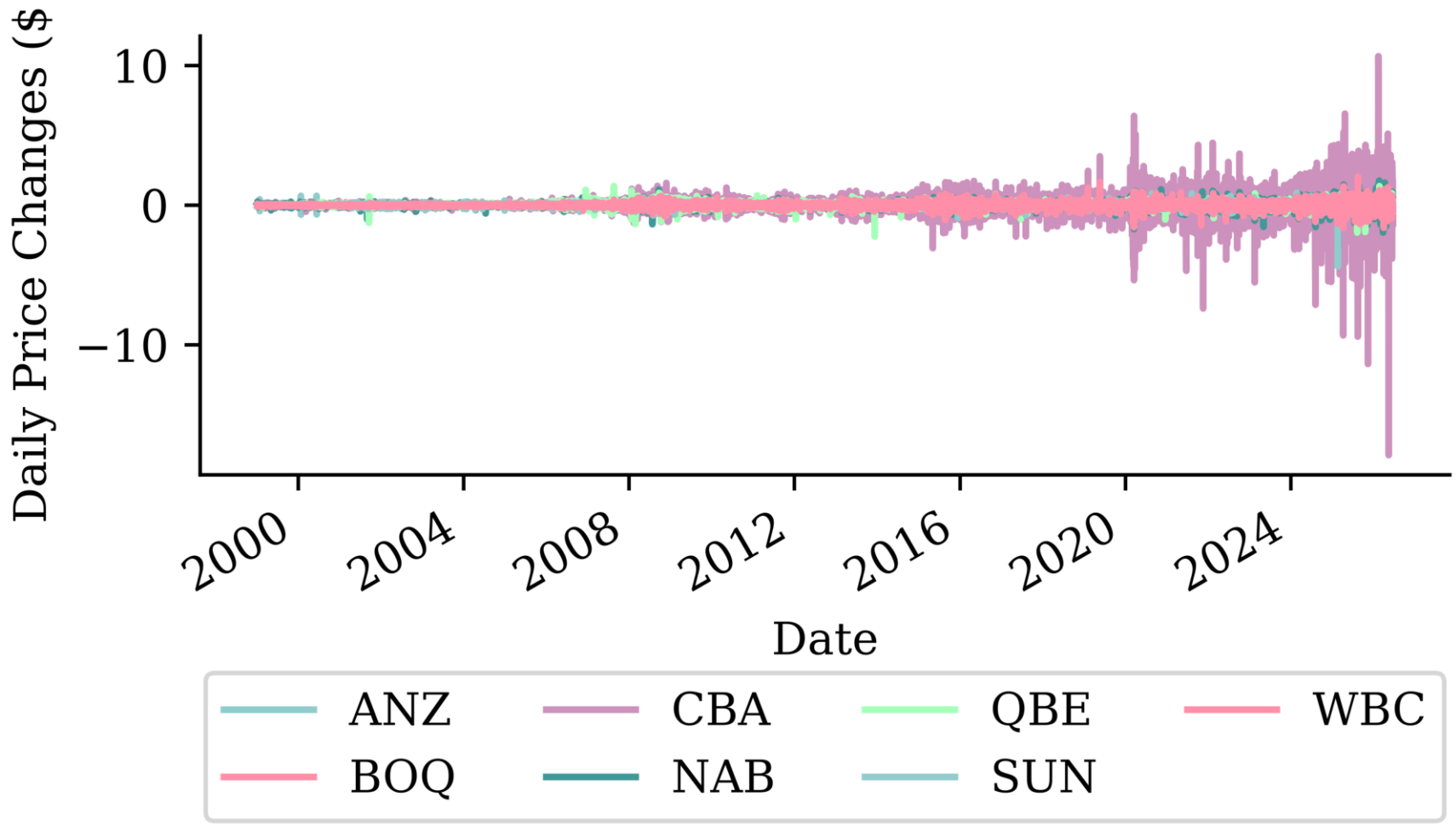
```
1 stocks.loc["2019-12":"2020", ["CBA"]]
```

	CBA
Date	
2019-12-02	64.326653
2019-12-03	62.675636
2019-12-04	61.467003
...	...
2020-12-29	68.825630
2020-12-30	68.481514
2020-12-31	67.269035

275 rows × 1 columns

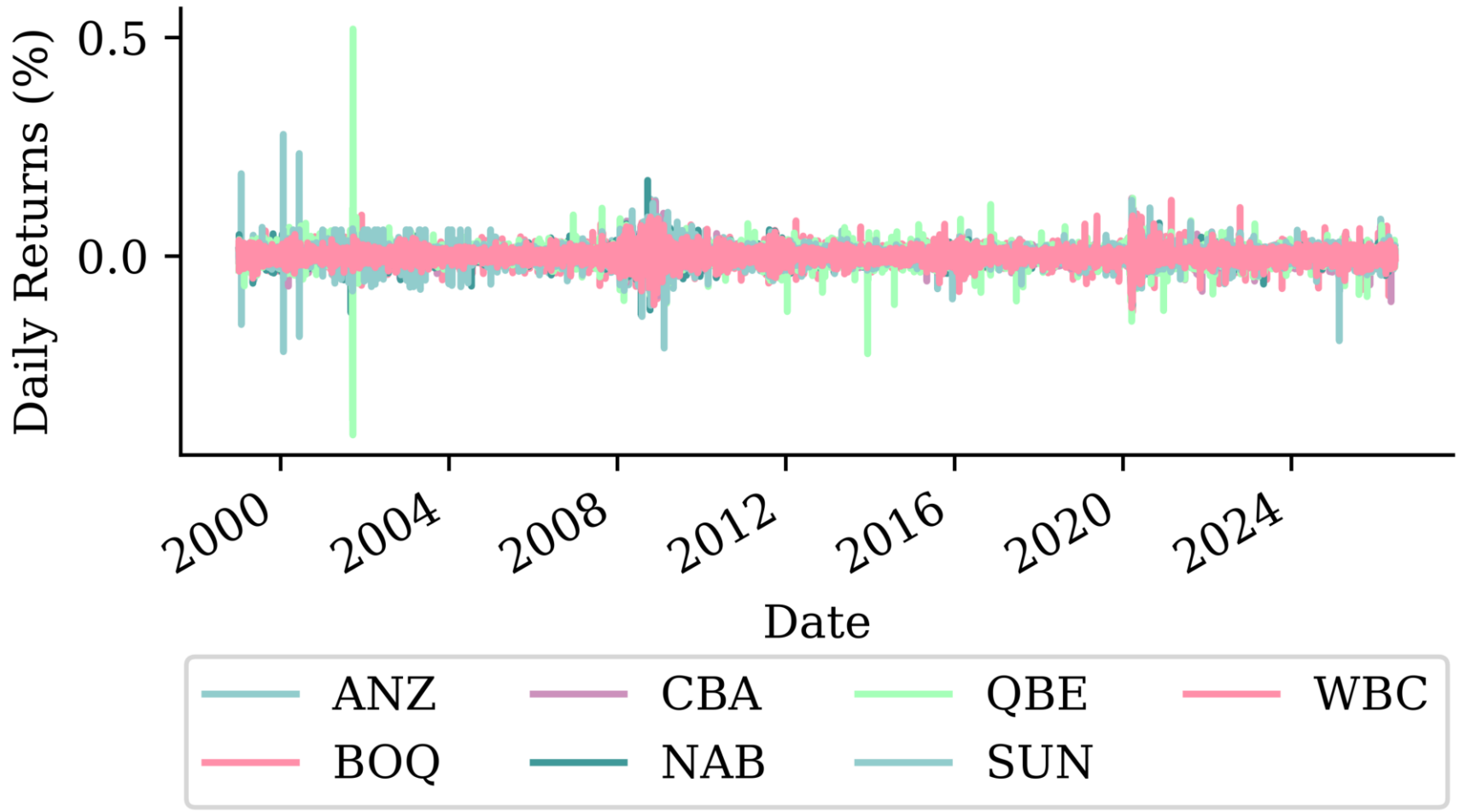
Can look at the first differences

```
1 stocks.diff().plot()
2 plt.ylabel("Daily Price Changes ($)")
3 plt.legend(loc="upper center", bbox_to_anchor=(0.5, -0.4), ncol=4);
```



Can look at the percentage changes

```
1 stocks.pct_change().plot()
2 plt.ylabel("Daily Returns (%)")
3 plt.legend(loc="upper center", bbox_to_anchor=(0.5, -0.4), ncol=4);
```



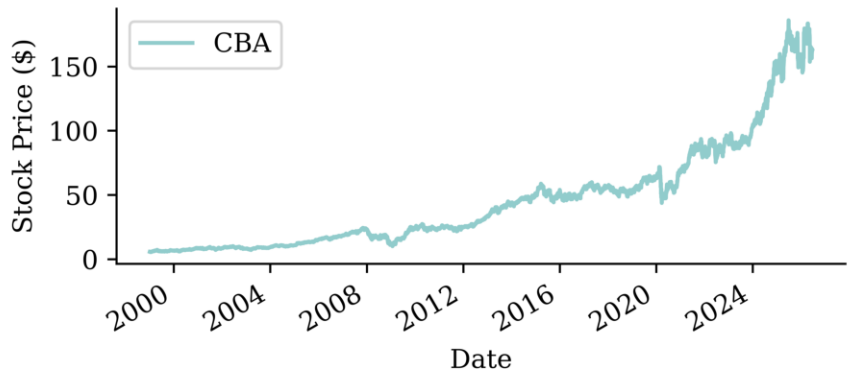
Focus on one stock

```
1 stock = stocks[["CBA"]].copy()
2 stock
```

	CBA
Date	
1999-01-01	5.823670
1999-01-04	5.760781
1999-01-05	5.763297
...	...
2026-06-17	163.710007
2026-06-18	162.229996
2026-06-19	162.399994

6993 rows × 1 columns

```
1 stock.plot()
2 plt.ylabel("Stock Price ($)");
```




```
1 stock.isna().sum()
```

```
CBA    0
dtype: int64
```

Fill in the missing values

```
asx200 = pd.DataFrame(asx200).set_index(stocks.index)
missing_day = asx200.index[asx200["ASX200"].isna()][1]
prev_day = missing_day - pd.Timedelta(days=1)
after = missing_day + pd.Timedelta(days=3)
```

```
asx200.loc[prev_day:after]
```

	ASX200
Date	
1999-01-25	2713.100098
1999-01-26	NaN
1999-01-27	2738.800049
1999-01-28	2765.100098
1999-01-29	2781.699951

```
asx200 = asx200.ffill()
asx200.loc[prev_day:after]
```

	ASX200
Date	
1999-01-25	2713.100098
1999-01-26	2713.100098
1999-01-27	2738.800049
1999-01-28	2765.100098
1999-01-29	2781.699951

```
stock = stock.ffill()
```

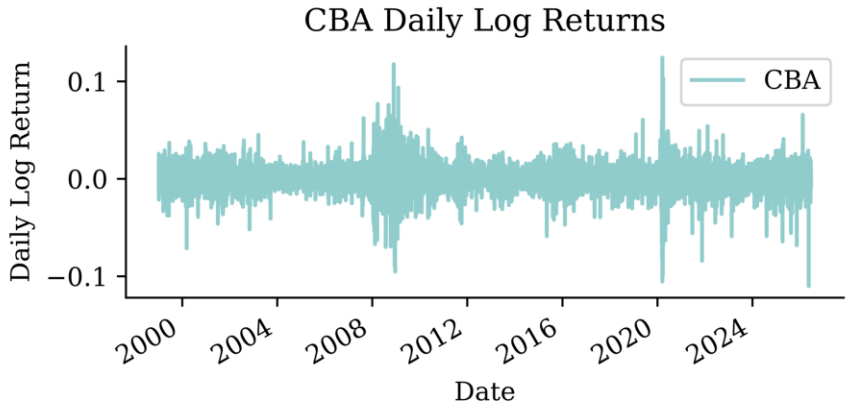
UNSW SYDNEY

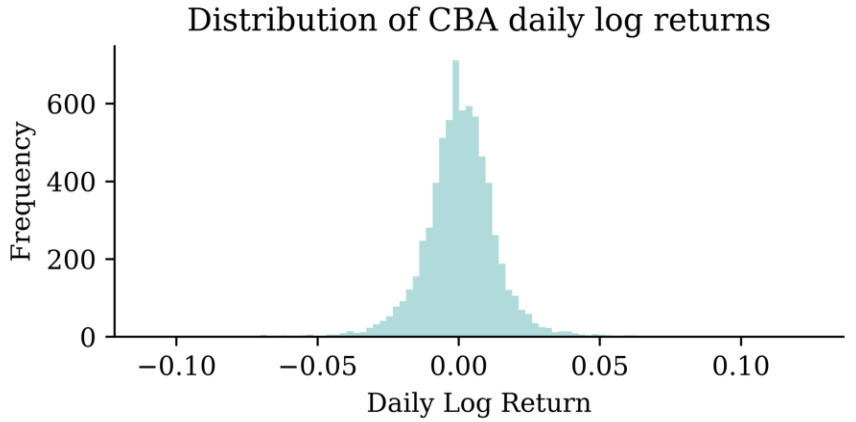
Convert to log returns

Instead of working with raw prices, we'll work with daily log returns:

```
# Calculate log returns
stock_log = np.log(stock / stock.shift(1)
stock_log.head()
```

Date	CBA
1999-01-04	-0.010858
1999-01-05	0.000437
1999-01-06	0.008043
1999-01-07	0.025859
1999-01-08	-0.021322



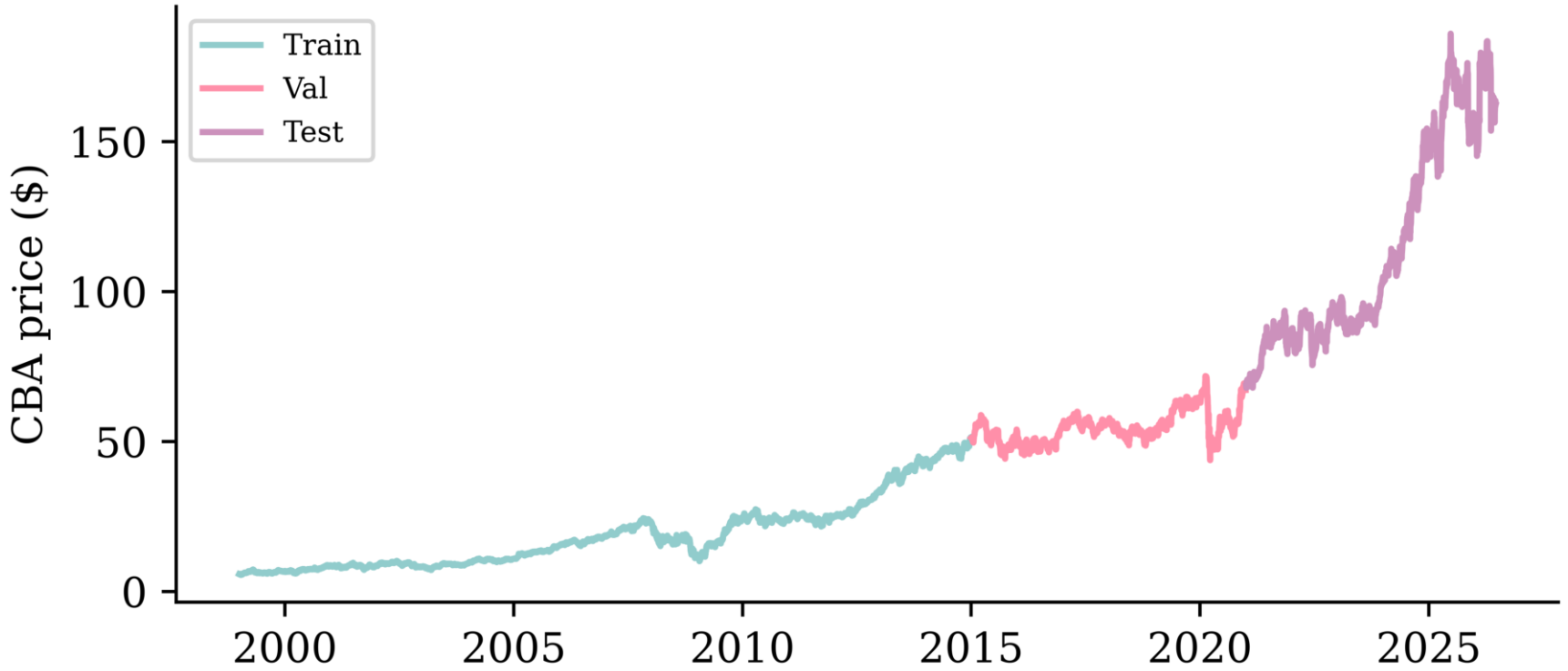


UNSW SYDNEY

Splitting in time

We never shuffle a time series: train on the oldest data, validate on the middle, and keep the most recent years to test on. The cutoffs live in one place — change these lines to re-split.

```
# Train through *_train_end, validate through *_val_end, test after.
cba_train_end,  cba_val_end  = "2014", "2020"   # ~60/20/20 (finance data ends 2026)
cba_val_start,  cba_test_start  = str(int(cba_train_end) + 1),  str(int(cba_val_end) + 1)
```



Lecture Outline

- Introduction
- Time series data
- Australian financial stocks
- **Sydney weather**
- Naive baseline forecasts
- Use the recent history
- Simple Recurrent Neural Networks
- Recurrent Neural Networks
- Multi-step forecasting and multivariate forecasting
- Car Crash NLP with RNNs

Sydney Airport weather

The Bureau of Meteorology records daily weather at Sydney Airport (station 066037).

```
TMAX = "Maximum temperature in 24 hours after 9am (local time) in Degrees C"
weather = pd.read_csv("data/raw/BoM/DC02D_Data_066037_9999999910249598.csv", low_memory=False)
weather
```

	dc	Station Number	Year	Month	Day	Precipitation in the 24 hours before 9am (local time) in mm	Quality of precipitation value	Number of days of rain within the days of accumulation	Accumulated number of days over which the precipitation was measured	Precipitation since last observation at 00 hours Local Time in mm	...
0	dc	66037	1991	1	1	0.0	Y				...
1	dc	66037	1991	1	2	0.2	Y	1	1		...
2	dc	66037	1991	1	3	0.0	Y				...
...	...	...	...	...	...	...	...	...	...	...	...
11655	dc	66037	2022	11	29	0.2	N		1	0.0	...
11656	dc	66037	2022	11	30	0.2	N		1	0.0	...
11657	dc	66037	2022	12	1	0.2	N		1	0.0	...

11658 rows × 120 columns

The maximum temperature series

We forecast the *daily maximum temperature*. First build a proper date index.

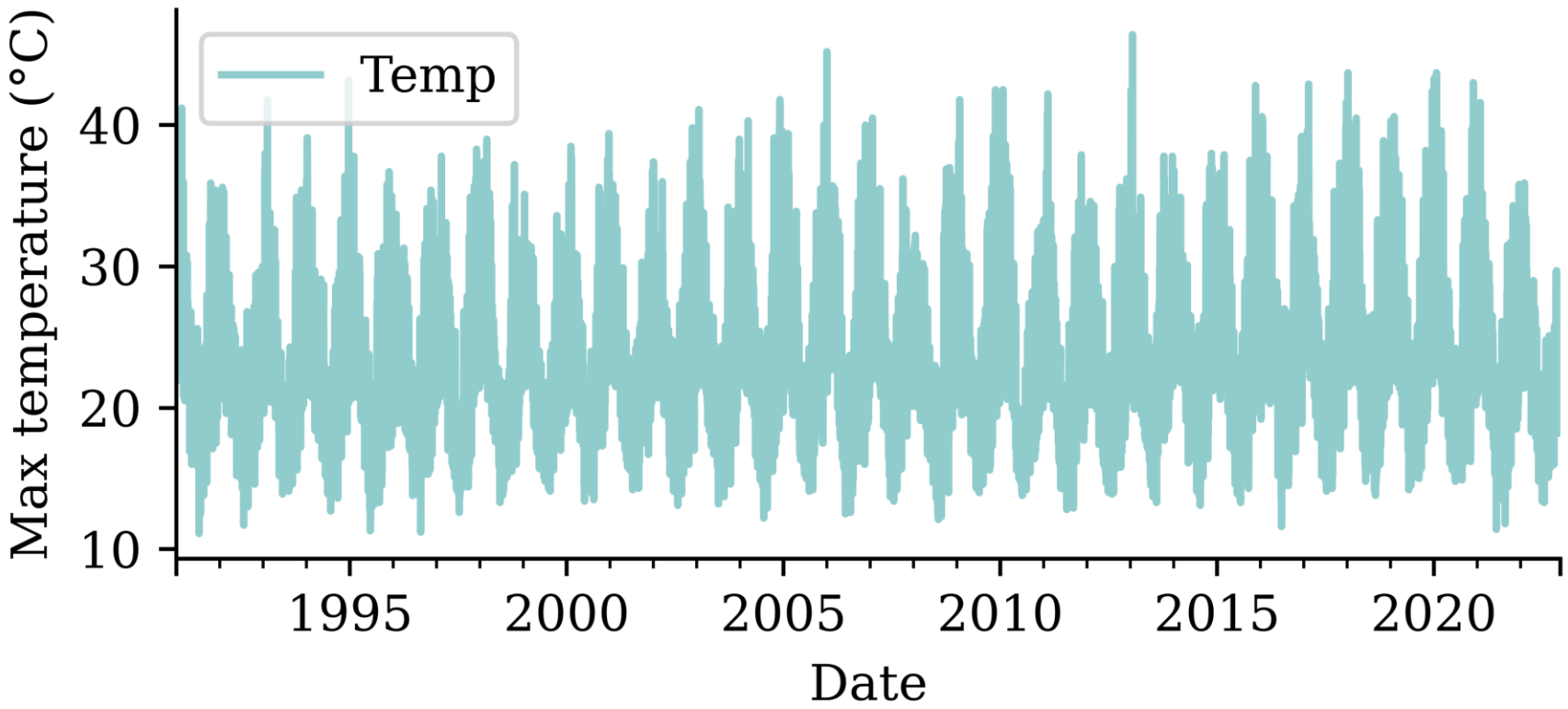
```
weather["Date"] = pd.to_datetime(weather[["Year", "Month", "Day"]])
temps = weather.set_index("Date")[[TMAX]].rename(columns={TMAX: "Temp"})
temps["Temp"] = pd.to_numeric(temps["Temp"], errors="coerce")
temps
```

	Temp
Date	
1991-01-01	28.0
1991-01-02	29.5
1991-01-03	31.2
...	...
2022-11-29	24.0
2022-11-30	21.8
2022-12-01	NaN

11658 rows × 1 columns

Plot

```
1 temps.plot()
2 plt.ylabel("Max temperature (°C)");
```



Data types and NA values

```
1 temps.info()
```

```
<class 'pandas.DataFrame'>
DatetimeIndex: 11658 entries, 1991-01-01 to
2022-12-01
Data columns (total 1 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Temp    11657 non-null  float64
dtypes: float64(1)
memory usage: 182.2 KB
```

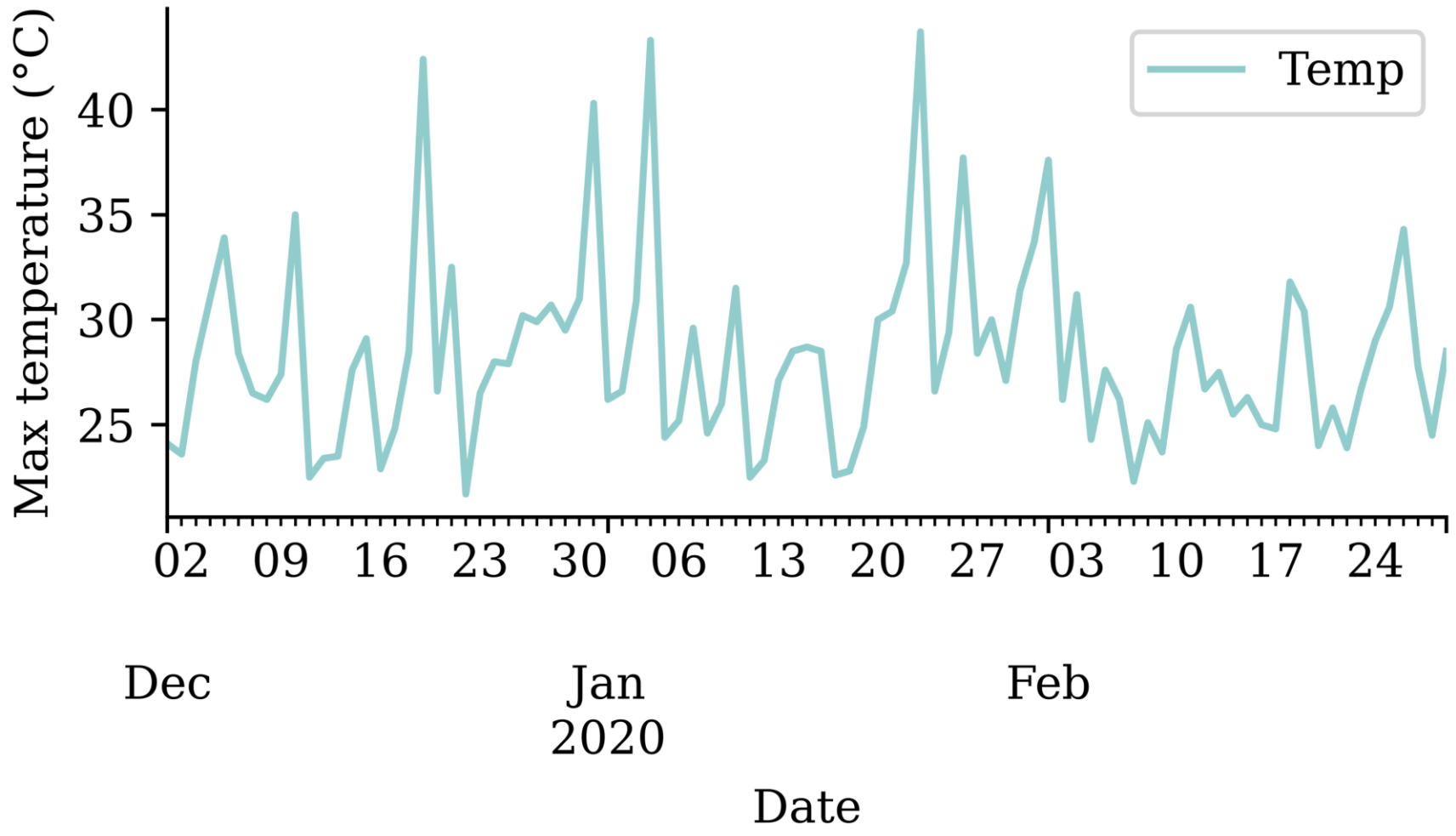
```
1 temps.isna().sum()
```

```
Temp    1
dtype: int64
```

```
1 temps = temps.dropna()
```

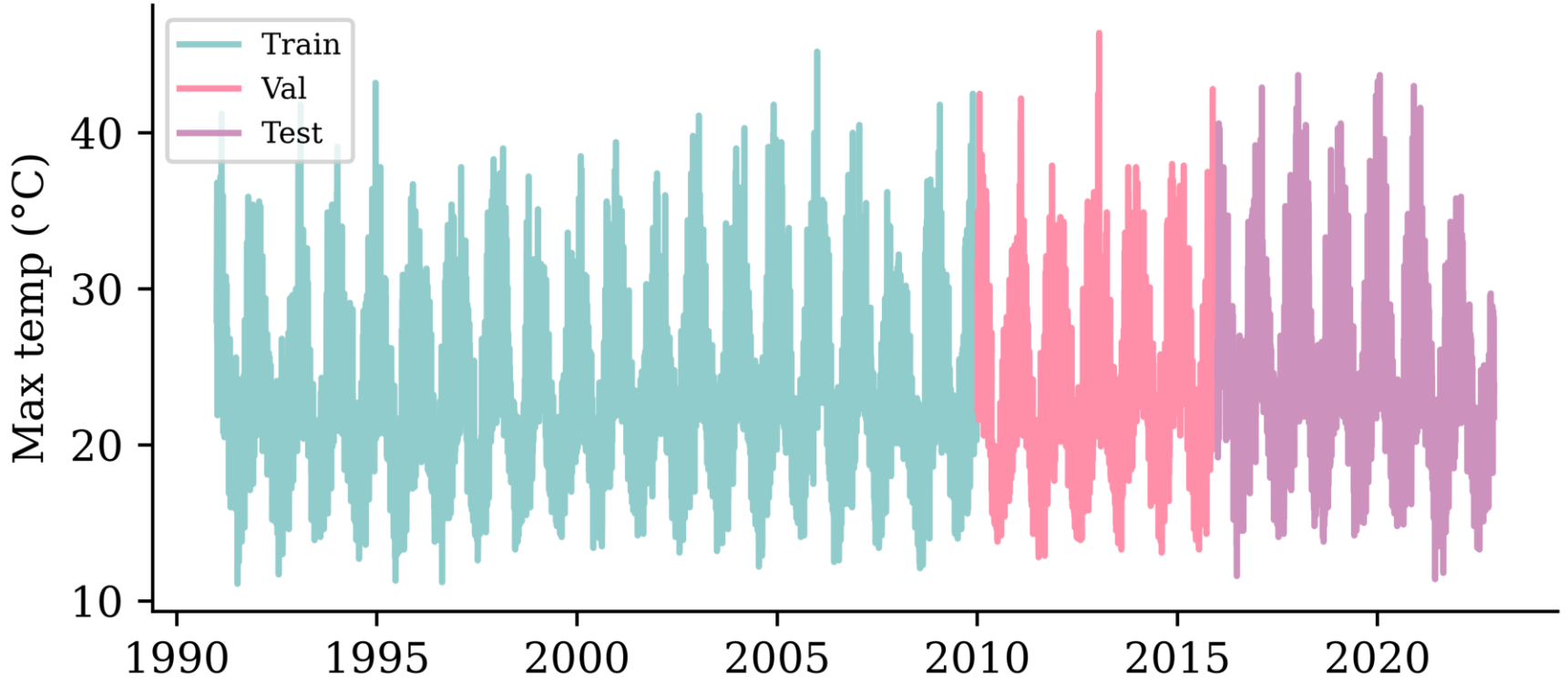
Zooming into one summer

```
1 temps.loc["2019-12":"2020-02"].plot()
2 plt.ylabel("Max temperature (°C)");
```



Splitting in time

```
temp_train_end, temp_val_end = "2009", "2015"    # ~60/20/20 (weather data ends 2022)
temp_val_start, temp_test_start = str(int(temp_train_end) + 1), str(int(temp_val_end) + 1)
```



Lecture Outline

- Introduction
- Time series data
- Australian financial stocks
- Sydney weather
- **Naive baseline forecasts**
- Use the recent history
- Simple Recurrent Neural Networks
- Recurrent Neural Networks
- Multi-step forecasting and multivariate forecasting
- Car Crash NLP with RNNs

Three naive forecasting ideas

The simplest forecast assumes *nothing changes* — but that has two readings, differing in how far ahead they look and where they re-anchor:

- *Persistence* (one-step): tomorrow equals today, $\hat{y}_t = y_{t-1}$, re-anchored to the *true* latest value every day. The yardstick for *one-step* models.
- *Constant* (multi-step): the entire future is held *constant* at the last value we saw, $\hat{y}_{t+h} = y_t$ for all h — anchored *once* at the forecast origin, giving a flat line. The yardstick for *multi-step* models.

Both are the same *random-walk* idea (also called *last observation carried forward*): they agree on the first step and only diverge after it.

The last basic forecast is to just extrapolate the trend seen in the training data.

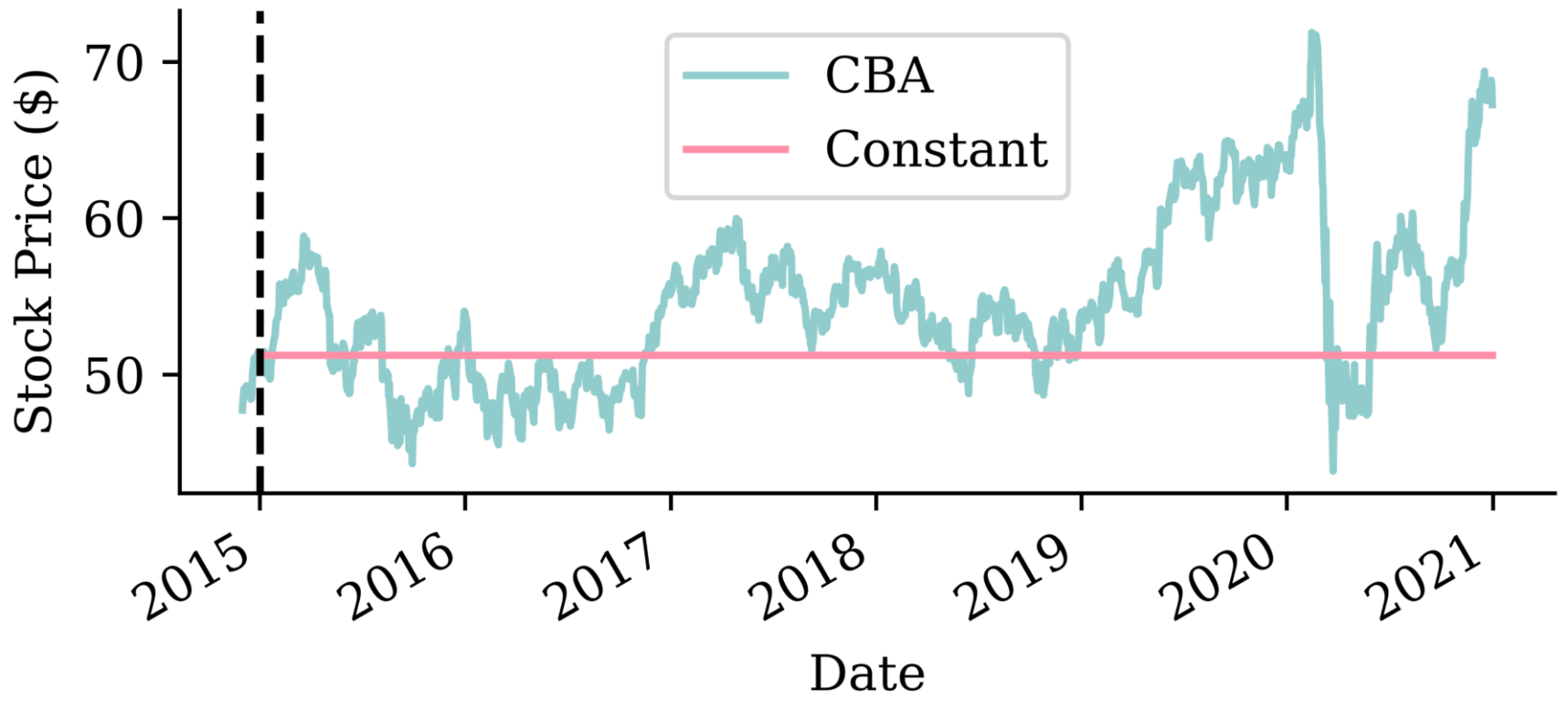
Helper functions

```python
def log_to_price(log_returns, initial_price):
    """Convert log returns to raw prices given an initial price."""
    # Use cumulative sum of log returns for numerical stability
    # P_t = P_0 * exp(sum of log returns from 1 to t)
    cumulative_log_returns = log_returns.cumsum()
    prices = initial_price * np.exp(cumulative_log_returns)

    return prices

def get_last_price(stock_df, cutoff_date):
    """Get the last known price before the forecast period starts."""
    last_known_date = stock_df.loc[:cutoff_date].index[-1]
    return stock_df.loc[last_known_date, "CBA"]
```

Constant forecast

The whole validation period is held *constant* at the last price we saw just before it. This is the multi-step naïve forecast (zero log returns from the origin onward).

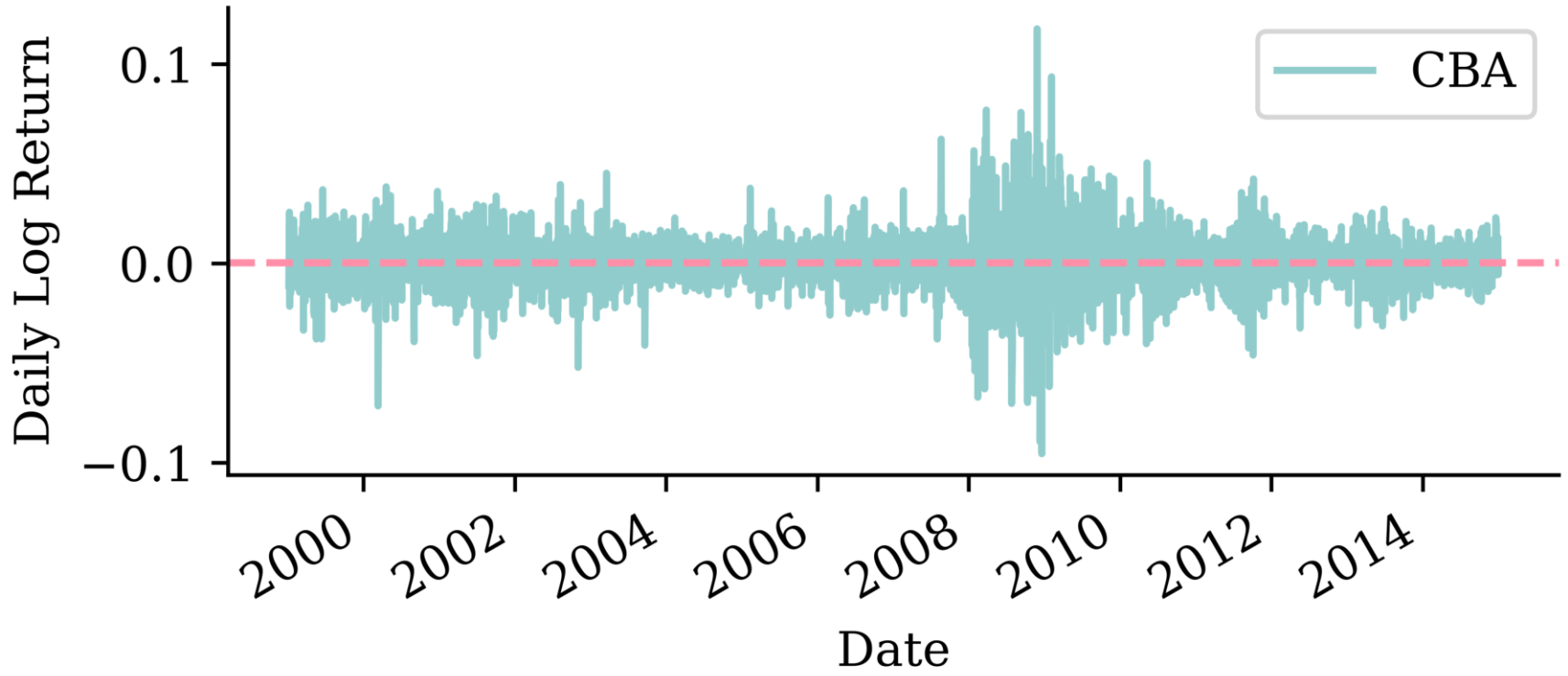
```
last_price = get_last_price(stock, cutoff_date=f"{cba_train_end}-12")
constant_log = pd.Series(0.0, index=stock_log.loc[cba_val_start:cba_val_end].index)
constant_prices = log_to_price(constant_log, last_price)
```



Extrapolate the trend

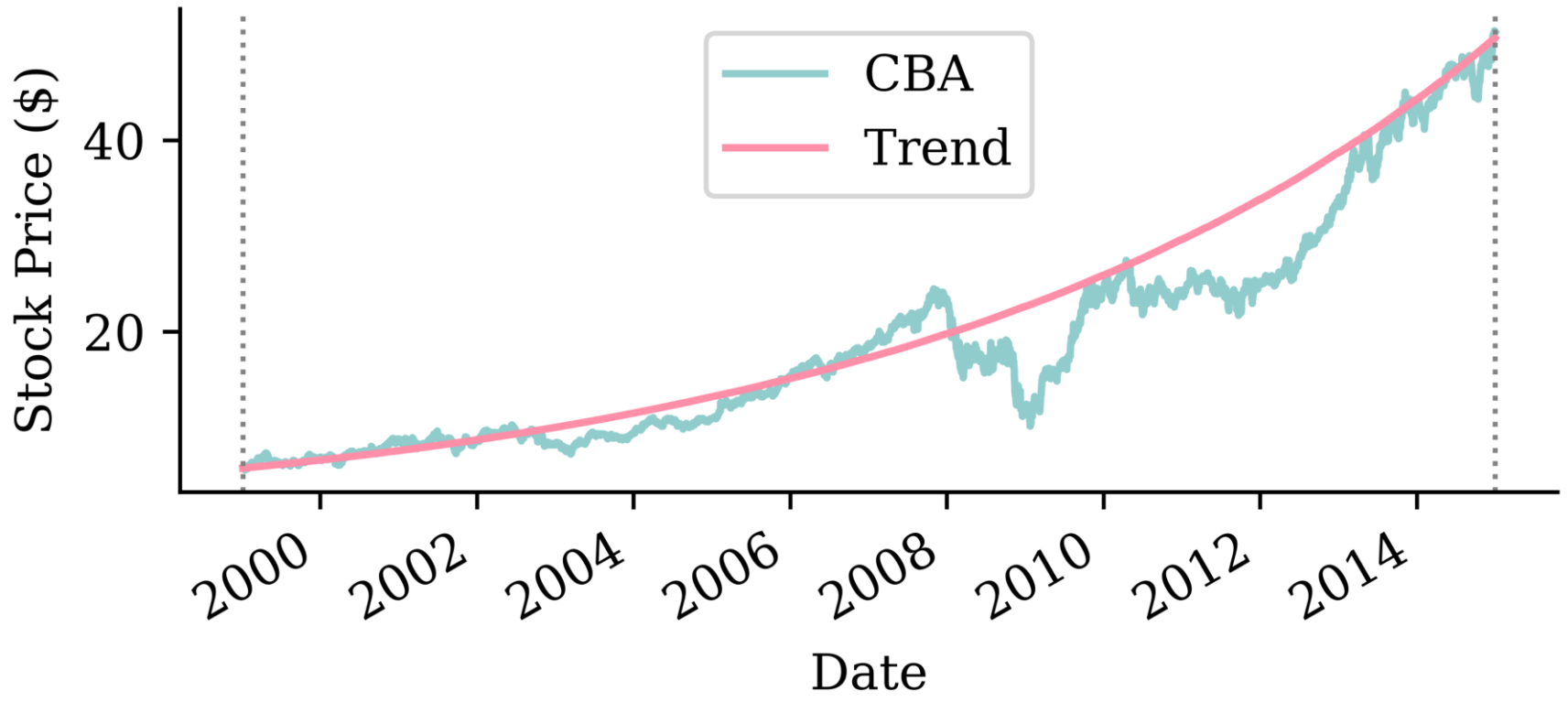
```
train_log_returns = stock_log.loc[:cba_train_end]   # estimate the trend on training data
trend_log = train_log_returns.mean().values[0]
print(f"Average daily log return: {trend_log:.6f}")
```

```
Average daily log return: 0.000532
```

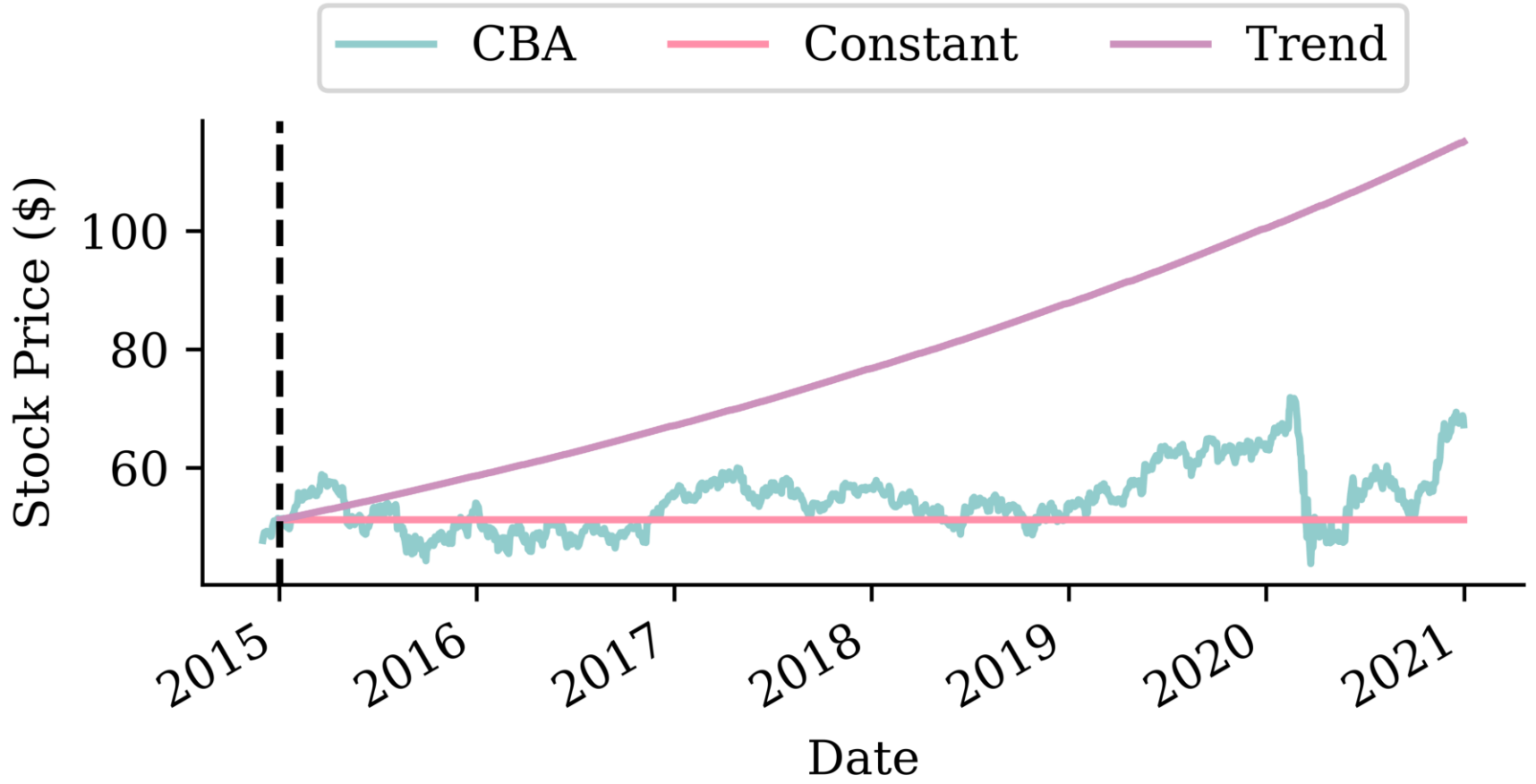


Trend fitted

```
# Create trend forecast over the training period to show the fitted trend
train_trend_log = pd.Series(trend_log, index=train_log_returns.index)
trend_start_price = get_last_price(stock, cutoff_date=train_log_returns.index[0].strftime
train_trend_prices = log_to_price(train_trend_log, trend_start_price)
```



Trend forecasts



Which is better?

If we look at the root mean squared error (RMSE) of the two models:

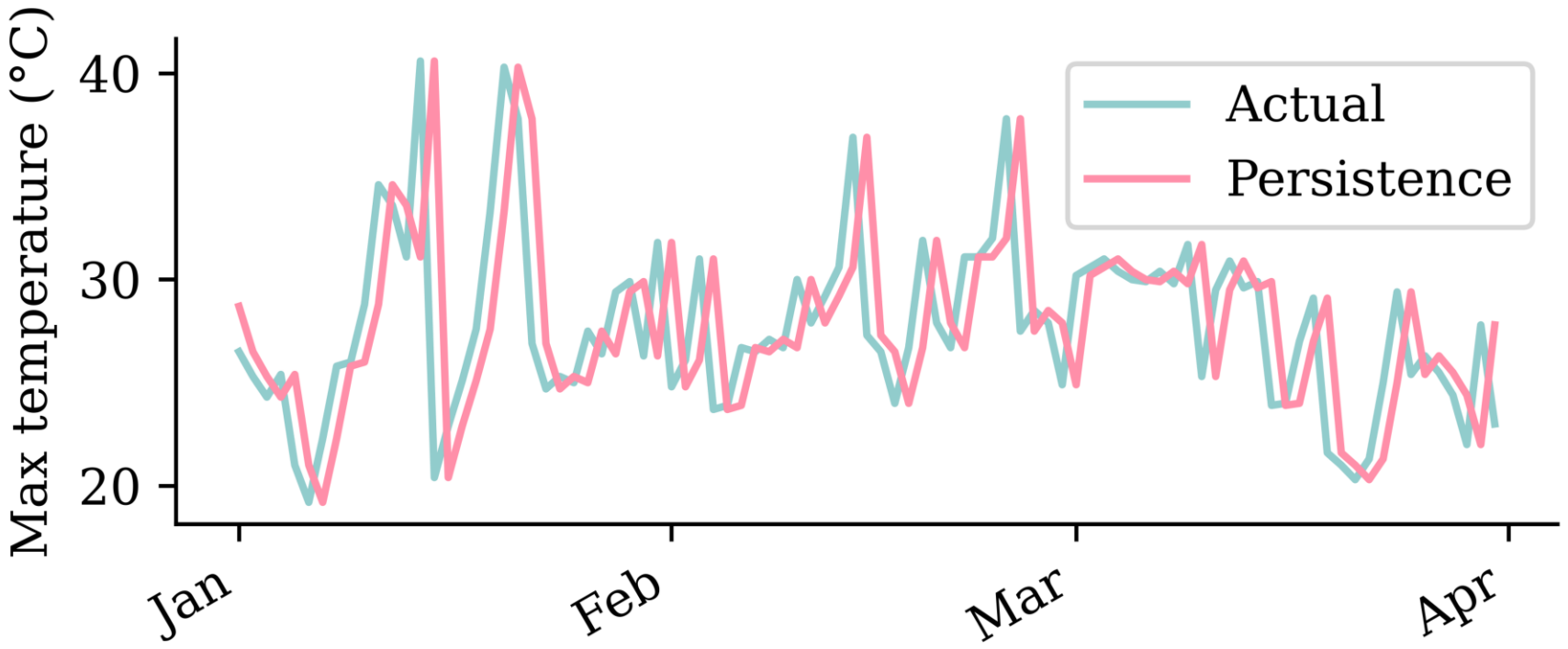
```
# Calculate RMSE using the actual forecasts we computed
actual_prices = stock.loc[cba_val_start:cba_val_end, "CBA"]
constant_rmse = mean_squared_error(actual_prices, constant_prices) ** 0.5
trend_rmse = mean_squared_error(actual_prices, trend_prices) ** 0.5
constant_rmse, trend_rmse
```

```
(6.190991093515316, 29.04712662478564)
```

Persistence forecast (one-step)

Now two baselines for the *temperature* series. First, *persistence*: predict tomorrow's maximum temperature to be the *same as today's*, $\hat{y}_t = y_{t-1}$.

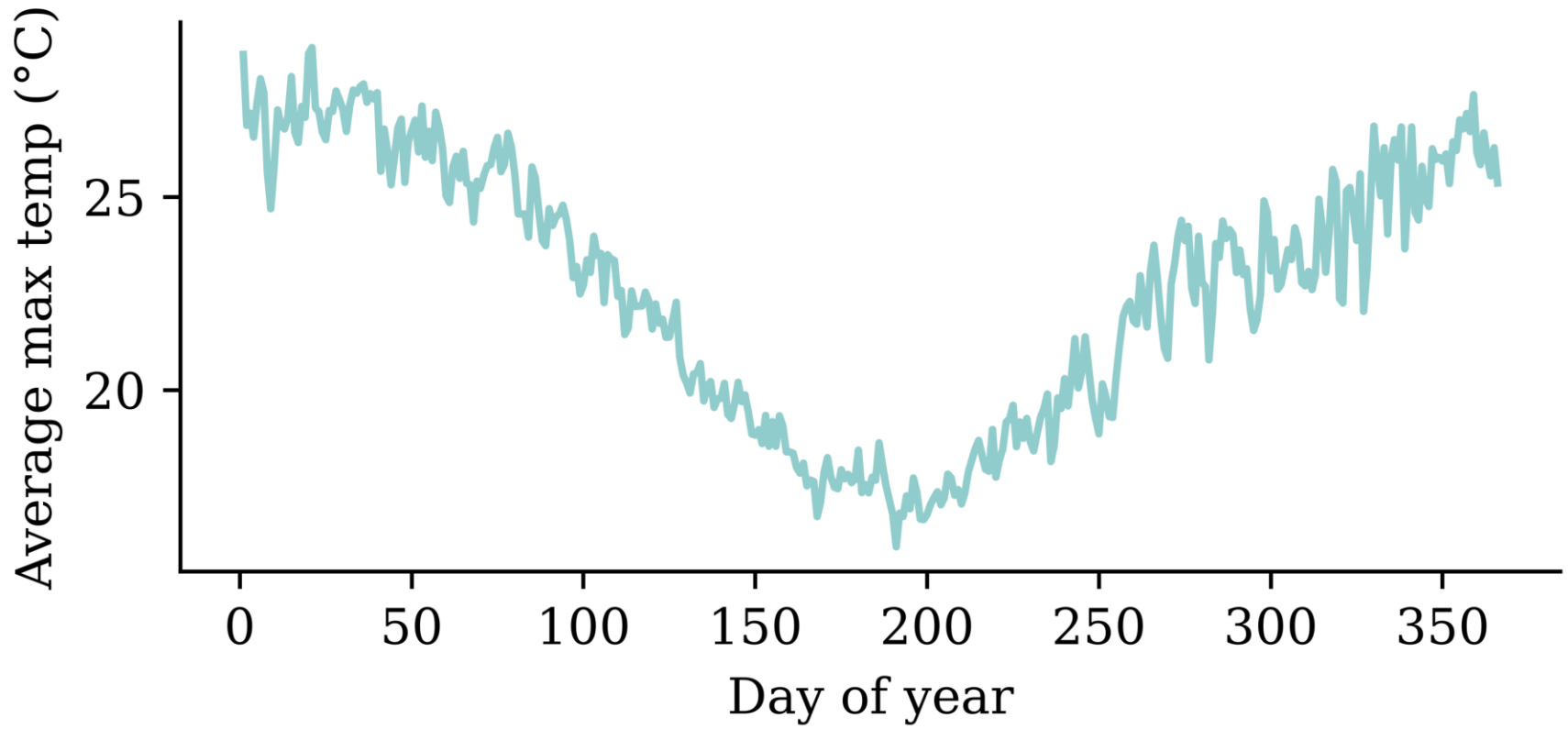
```
1  persistence = temps["Temp"].shift(1)
```



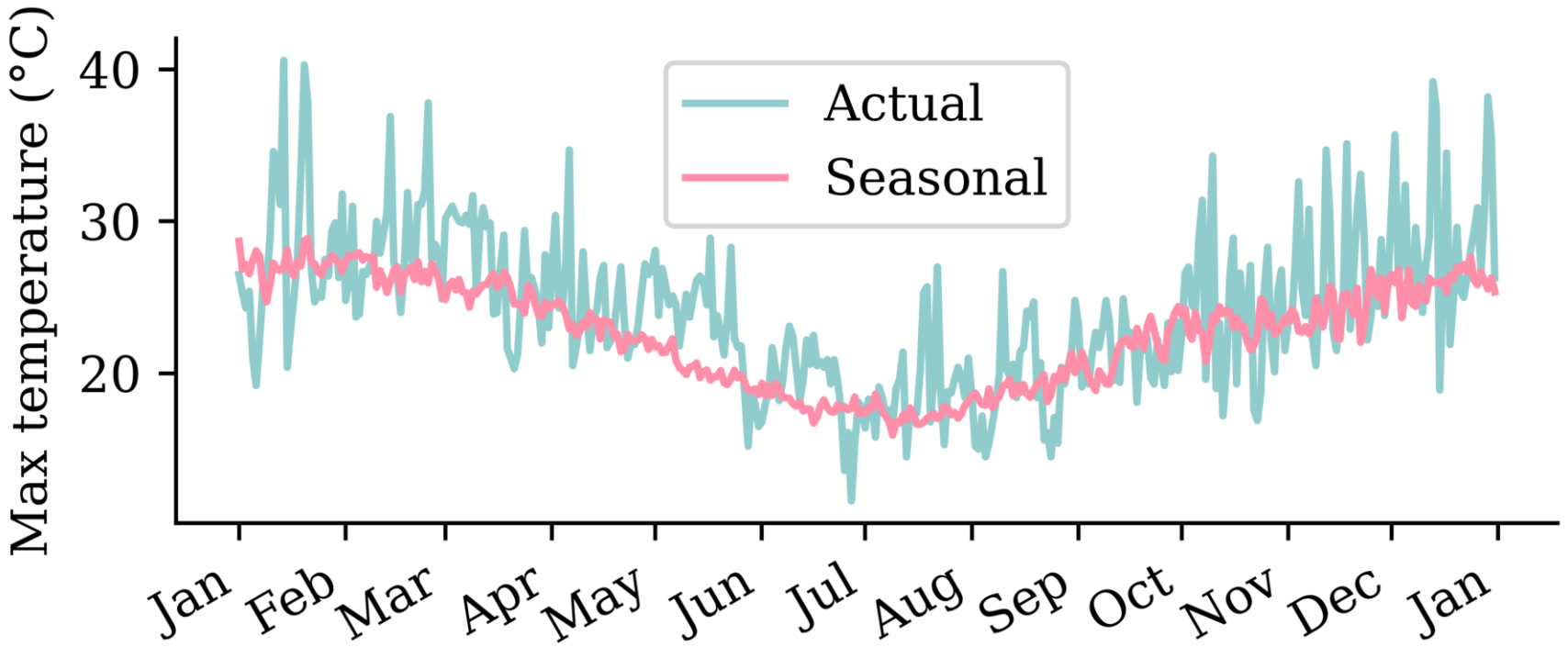
A seasonal baseline

A smarter baseline uses the *seasonality*: predict each day's historical average over all years in the training period.

```
train_temps = temps.loc[:temp_train_end, "Temp"]
seasonal_average = train_temps.groupby(train_temps.index.dayofyear).mean()
seasonal = pd.Series(temps.index.dayofyear, index=temps.index).map(seasonal_average)
```



Seasonal forecast



Which is better?

We compare the forecasts over the *validation* years using the root mean squared error (RMSE), in °C.

```
actual_val = temps["Temp"].loc[temp_val_start:temp_val_end]
persistence_rmse_val = mean_squared_error(actual_val, persistence.loc[temp_val_start:temp
seasonal_rmse_val = mean_squared_error(actual_val, seasonal.loc[temp_val_start:temp_val_e
persistence_rmse_val, seasonal_rmse_val
```

```
(4.069279101425016, 3.8466517052843585)
```

Lecture Outline

- Introduction
- Time series data
- Australian financial stocks
- Sydney weather
- Naive baseline forecasts
- **Use the recent history**
- Simple Recurrent Neural Networks
- Recurrent Neural Networks
- Multi-step forecasting and multivariate forecasting
- Car Crash NLP with RNNs

Lagged features

We turn each series into a table: the previous 40 values become the features `T-40, …, T-1`, and the value to predict is `T`.

```python
def lagged_timeseries(df, target, window):
    lagged = pd.DataFrame()
    for i in range(window, 0, -1):
        lagged[f"T-{i}"] = df[target].shift(i)
    lagged["T"] = df[target].values
    return lagged
```

```python
temp_lagged = lagged_timeseries(temps, "Temp", 40)
cba_lagged = lagged_timeseries(stock_log, "CBA", 40)
```

The temperature design matrix — each row is one 40-day window (the first rows include `NaN` until 40 days of history exist):

	T-40	T-39	T-38	T-37	T-36	T-35	T-34	T-33	T-32	T-31	…	T-9	T-8	T-7	T-6	T-5	T-4	T-3	T-2
Date																			
1991-01-01	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	…	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1991-01-02	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	…	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…
2022-11-29	24.3	23.9	25.3	21.1	21.3	27.2	29.6	28.5	25.7	27.2	…	27.7	24.5	23.5	28.6	24.7	25.1	22.8	28.2
2022-11-30	23.9	25.3	21.1	21.3	27.2	29.6	28.5	25.7	27.2	24.1	…	24.5	23.5	28.6	24.7	25.1	22.8	28.2	23.3

11657 rows × 41 columns

Building the design matrix

We turn each split into a supervised table of lagged features. We can't shuffle — the timing matters — so we fit on older data and forecast forward. The same recipe applies to both series.

Weather — predict the temperature (time-ordered split; cutoffs set above).

```
X_train_temp = temp_lagged.loc[:temp_train_end].dropna()
X_val_temp = temp_lagged.loc[temp_val_start:temp_val_end].dr
X_test_temp = temp_lagged.loc[temp_test_start:].dropna()
y_train_temp = X_train_temp.pop("T")
y_val_temp = X_val_temp.pop("T")
y_test_temp = X_test_temp.pop("T")
```

Finance — predict the log return (same time-ordered split; cutoffs set above).

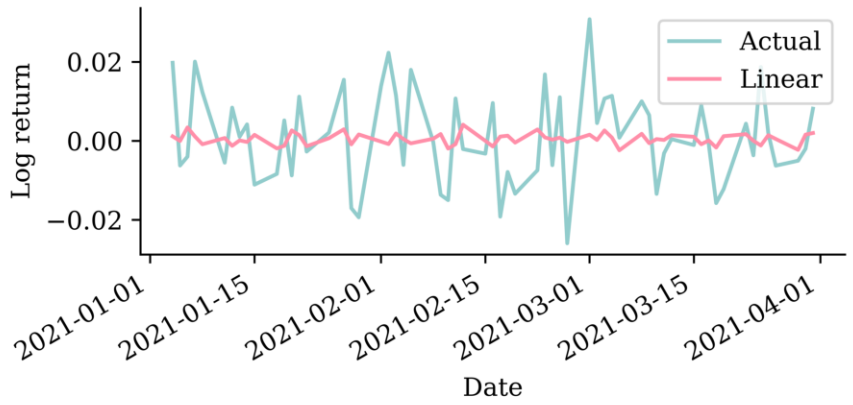
```
X_train_cba = cba_lagged.loc[:cba_train_end].dropna()
X_val_cba = cba_lagged.loc[f"{cba_val_start}-01":f"{cba_val_
X_test_cba = cba_lagged.loc[cba_test_start:].dropna()
y_train_cba = X_train_cba.pop("T")
y_val_cba = X_val_cba.pop("T")
y_test_cba = X_test_cba.pop("T")
```

UNSW SYDNEY

A linear model

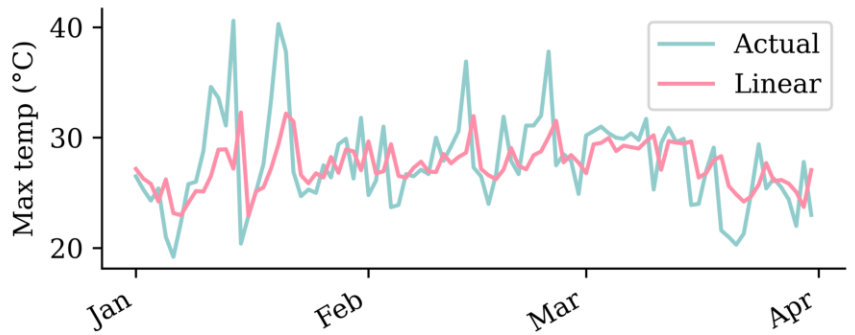
```
lr_s = LinearRegression().fit(X_train_cba, y_train_cba)
y_pred_cba = lr_s.predict(X_test_cba)
s_scores["Linear"] = s_rmse(y_pred_cba)
```

CBA log-return forecasts (Q1 of test set)




```
lr_w = LinearRegression().fit(X_train_temp, y_train_temp)
y_pred_temp = lr_w.predict(X_test_temp)
w_scores["Linear"] = w_rmse(y_pred_temp)
```

Temperature forecasts (Q1 of test set)



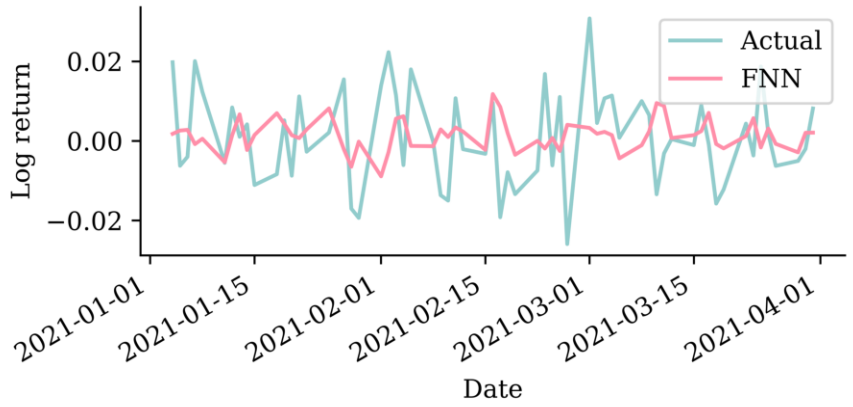
UNSW SYDNEY

A feedforward network

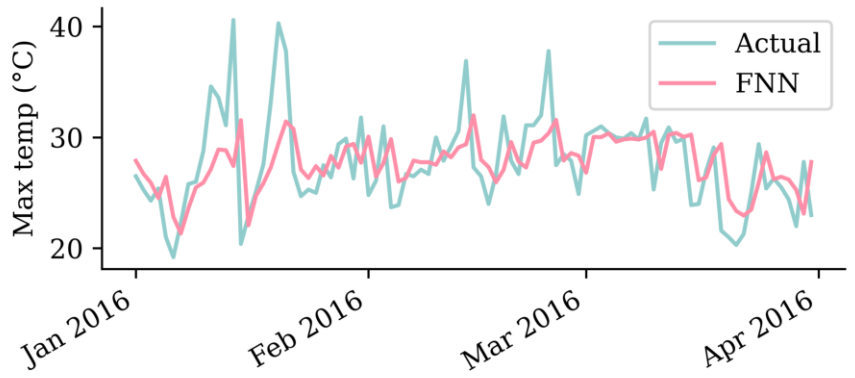
Same architecture for both; only the input rescaling differs: temperatures are divided by 40 (their rough maximum), log returns by their training-set standard deviation (`finance_scale` ≈ 0.014).

```
scale = 1 / finance_scale  # for weather: 1/40
model = Sequential([Rescaling(scale), Dense(32, activation="leaky_relu"), Dense(1)])
model.compile(optimizer="adam", loss="mean_absolute_error")
model.fit(X_train_cba, y_train_cba, validation_data=(X_val_cba, y_val_cba), epochs=500,
    callbacks=[EarlyStopping(patience=15, restore_best_weights=True)], verbose=0)
```

CBA log-return forecasts (Q1 of test set)



Temperature forecasts (Q1 of test set)



UNSW SYDNEY

Lecture Outline

- Introduction
- Time series data
- Australian financial stocks
- Sydney weather
- Naive baseline forecasts
- Use the recent history
- **Simple Recurrent Neural Networks**
- Recurrent Neural Networks
- Multi-step forecasting and multivariate forecasting
- Car Crash NLP with RNNs

Introduction

- A recurrent neural network is a type of neural network that is designed to process sequences of data (e.g. time series, sentences).
- A recurrent neural network is any network that contains a recurrent layer.
- A recurrent layer is a layer that processes data in a sequence.
- An RNN can have one or more recurrent layers.
- Weights are shared over time; this allows the model to be used on arbitrary-length sequences.

On the name of RNNs:

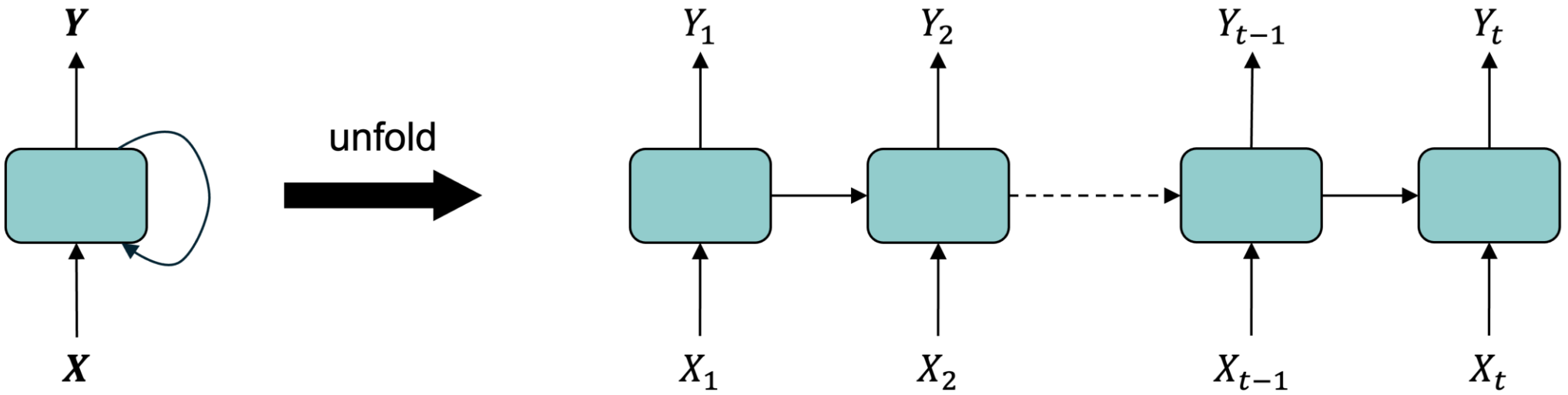
The "recurrent" comes from a *recurrence relation*: each element of a sequence is defined in terms of the previous one(s). When only the immediately preceding element matters, this looks like

$$u_n = \psi(n, u_{n-1}) \quad \text{for} \quad n > 0.$$

Example: Factorial $n! = n(n-1)!$ for $n > 0$ given $0! = 1$.

Diagram of an RNN cell

The RNN processes each data point in the sequence one by one, while keeping memory of what came before.

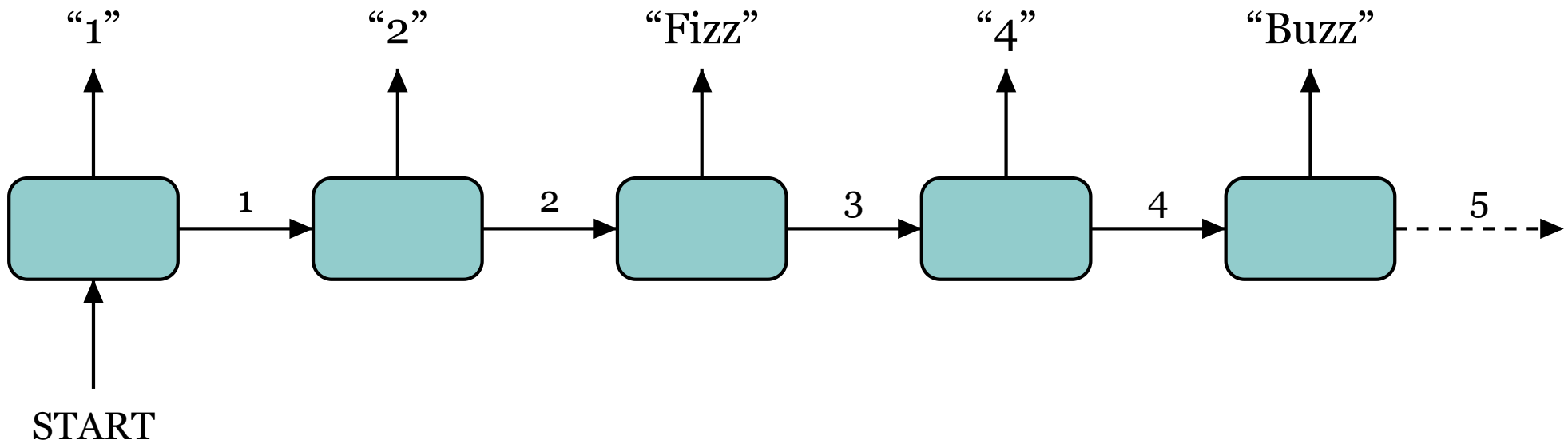


Schematic of a recurrent neural network. E.g. SimpleRNN, LSTM, or GRU.

Source: Melissa Renard (2025)

Intuition/demo: Fizz Buzz

Play *Fizz Buzz*: count up, but say "Fizz" on multiples of 3, "Buzz" on multiples of 5, and "Fizz Buzz" on multiples of both.



A vector-to-sequence RNN playing Fizz Buzz: one `START` input, the running count is the hidden state passed to the right, the spoken word is the output at each step.

Each player only needs the count from their neighbour (the hidden state) to know what to say and what to pass on — no separate input per step.

A SimpleRNN cell

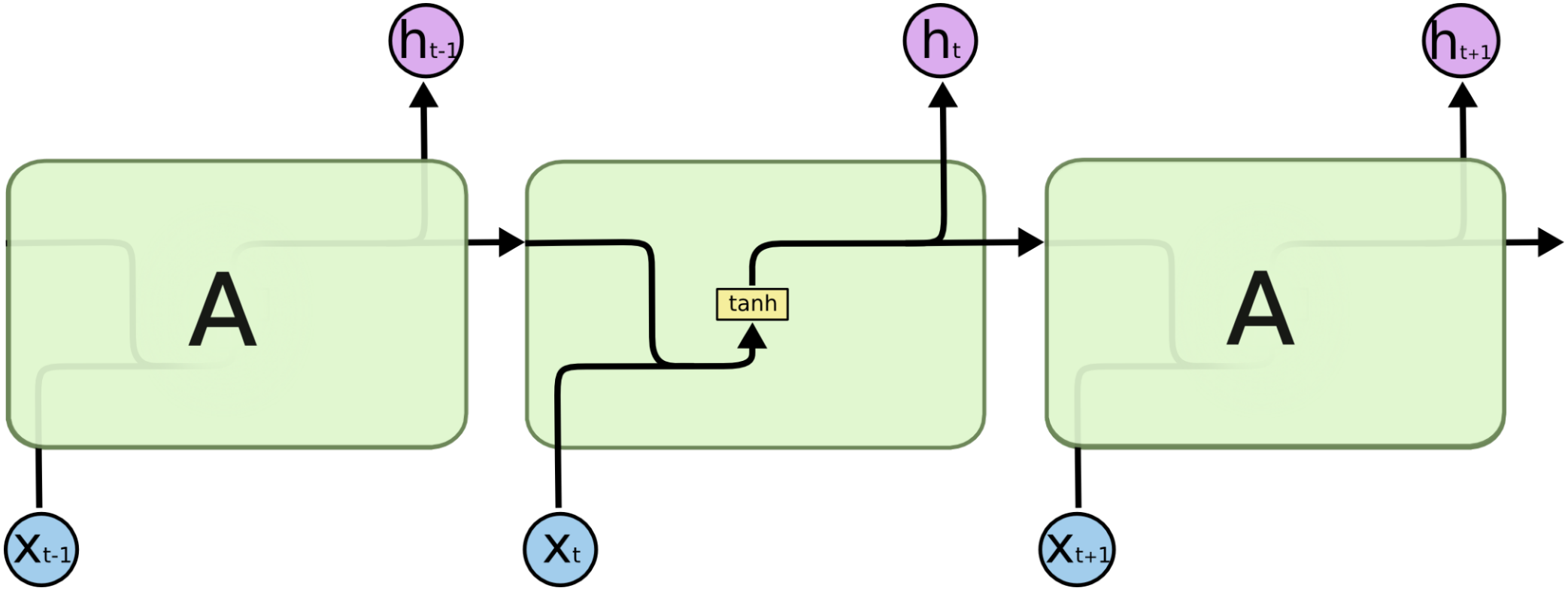


Diagram of a SimpleRNN cell.

All the outputs before the final one are often discarded.

Source: Christopher Olah (2015), Understanding LSTM Networks, Colah's Blog.

The rank of a time series

Say we had n observations of a time series $x_1, x_2, \ldots, x_n$.

This $\boldsymbol{x} = (x_1, \ldots, x_n)$ would have shape $(n,)$ & rank 1.

If instead we had a batch of b time series

$$\boldsymbol{X} = \begin{pmatrix} x_7 & x_8 & \cdots & x_{7+n-1} \\ x_2 & x_3 & \cdots & x_{2+n-1} \\ \vdots & \vdots & \ddots & \vdots \\ x_3 & x_4 & \cdots & x_{3+n-1} \end{pmatrix},$$

the batch $\boldsymbol{X}$ would have shape (b, n) & rank 2.

t	x	y
0	x_0	y_0
1	x_1	y_1
2	x_2	y_2
3	x_3	y_3

Say n observations of the m time series would be a shape (n, m) matrix of rank 2.

In Keras, a batch of b of these time series has shape (b, n, m) and has rank 3.

Note

Use $\boldsymbol{x}_t \in \mathbb{R}^{1\times m}$ to denote the vector of all time series at time t. Here, $\boldsymbol{x}_t = (x_t, y_t)$.

SimpleRNN

Say each prediction is a vector of size d, so $\boldsymbol{y}_t \in \mathbb{R}^{1\times d}$.

Then the main equation of a SimpleRNN, given $\boldsymbol{y}_0 = \mathbf{0}$, is

$$\boldsymbol{y}_t = \psi\big(\boldsymbol{x}_t \boldsymbol{W}_x + \boldsymbol{y}_{t-1}\boldsymbol{W}_y + \boldsymbol{b}\big).$$

Here,

$$\boldsymbol{x}_t \in \mathbb{R}^{1\times m}, \boldsymbol{W}_x \in \mathbb{R}^{m\times d},$$
$$\boldsymbol{y}_{t-1} \in \mathbb{R}^{1\times d}, \boldsymbol{W}_y \in \mathbb{R}^{d\times d}, \text{ and } \boldsymbol{b} \in \mathbb{R}^{d}.$$

SimpleRNN (in batches)

Say we operate on batches of size b, then $\boldsymbol{Y}_t \in \mathbb{R}^{b \times d}$.

The main equation of a SimpleRNN, given $\boldsymbol{Y}_0 = \boldsymbol{0}$, is

$$\boldsymbol{Y}_t = \psi\left(\boldsymbol{X}_t \boldsymbol{W}_x + \boldsymbol{Y}_{t-1} \boldsymbol{W}_y + \boldsymbol{b}\right).$$

Here,

$$\begin{aligned} &\boldsymbol{X}_t \in \mathbb{R}^{b \times m}, \boldsymbol{W}_x \in \mathbb{R}^{m \times d}, \\ &\boldsymbol{Y}_{t-1} \in \mathbb{R}^{b \times d}, \boldsymbol{W}_y \in \mathbb{R}^{d \times d}, \text{ and } \boldsymbol{b} \in \mathbb{R}^{d}. \end{aligned}$$

Remember, $\boldsymbol{X} \in \mathbb{R}^{b \times n \times m}$, $\boldsymbol{Y} \in \mathbb{R}^{b \times d}$, and $\boldsymbol{X}_t$ is equivalent to `X[:, t, :]`.

Simple Keras demo

```
num_obs = 4
num_time_steps = 3
num_time_series = 2

X = (
    np.arange(num_obs * num_time_steps * num_time_series)
    .astype(np.float32)
    .reshape([num_obs, num_time_steps, num_time_series])
)

output_size = 1
y = np.array([0, 0, 1, 1])
```

```
X[:2]
```

```
array([[[ 0.,  1.],
        [ 2.,  3.],
        [ 4.,  5.]],

       [[ 6.,  7.],
        [ 8.,  9.],
        [10., 11.]]], dtype=float32)
```

```
X[2:]
```

```
array([[[12., 13.],
        [14., 15.],
        [16., 17.]],

       [[18., 19.],
        [20., 21.],
        [22., 23.]]], dtype=float32)
```

Keras' SimpleRNN

As usual, the `SimpleRNN` is just a layer in Keras.

```
random.seed(1234)
model = Sequential([SimpleRNN(output_size, activation="sigmoid")])
model.compile(loss="binary_crossentropy", metrics=["accuracy"])

hist = model.fit(X, y, epochs=500, verbose=False)
model.evaluate(X, y, verbose=False)
```

```
[8.05906867980957, 0.5]
```

The predicted probabilities on the training set are:

```
model.predict(X, verbose=0)
```

```
array([[8.56e-05],
       [2.25e-10],
       [5.98e-16],
       [1.59e-21]], dtype=float32)
```

SimpleRNN weights

```
model.get_weights()
```

```
[array([[-1.47],
        [-0.67]], dtype=float32),
 array([[0.99]], dtype=float32),
 array([-0.14], dtype=float32)]
```

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))


W_x, W_y, b = model.get_weights()

Y = np.zeros((num_obs, output_size), dtype=np.float32)
for t in range(num_time_steps):
    X_t = X[:, t, :]
    z = X_t @ W_x + Y @ W_y + b
    Y = sigmoid(z)

Y
```

```
array([[8.56e-05],
       [2.25e-10],
       [5.98e-16],
       [1.59e-21]], dtype=float32)
```

Lecture Outline

- Introduction
- Time series data
- Australian financial stocks
- Sydney weather
- Naive baseline forecasts
- Use the recent history
- Simple Recurrent Neural Networks
- **Recurrent Neural Networks**
- Multi-step forecasting and multivariate forecasting
- Car Crash NLP with RNNs

Long short-term memory (LSTM)

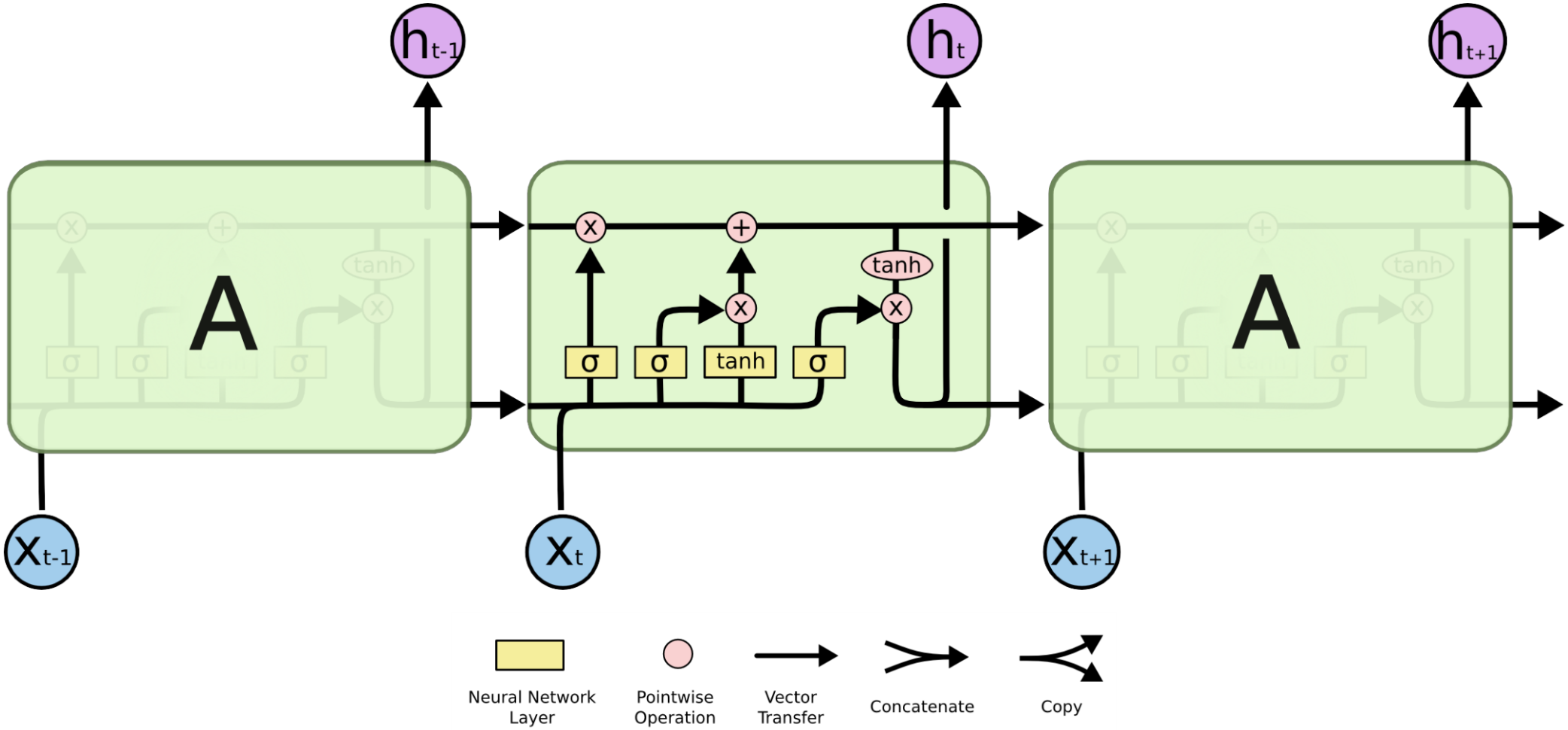
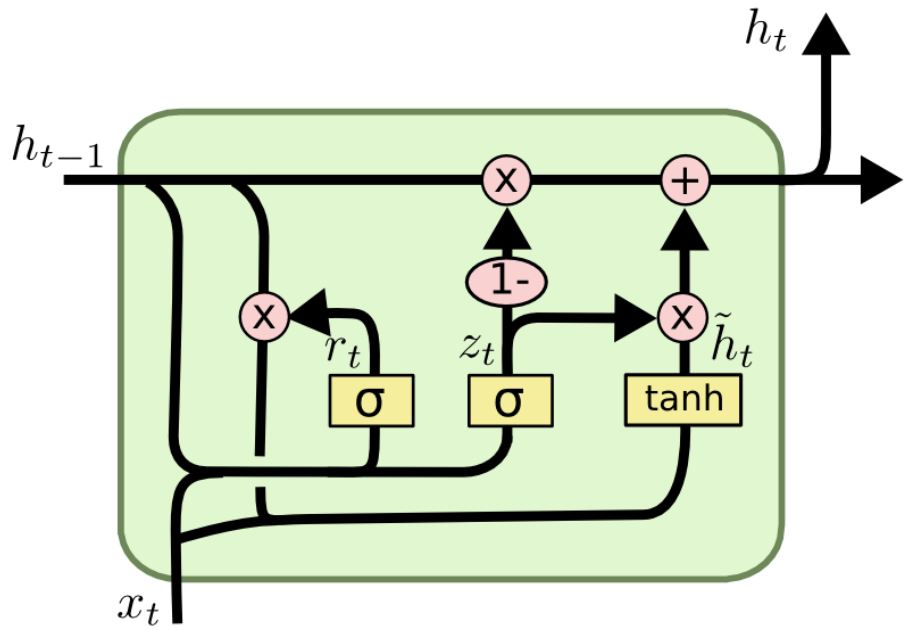


Diagram of an LSTM cell.

Sources: Hochreiter & Schmidhuber (1997), and Christopher Olah (2015), Understanding LSTM Networks, Colah's Blog.

Gated Recurrent Unit (GRU)



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

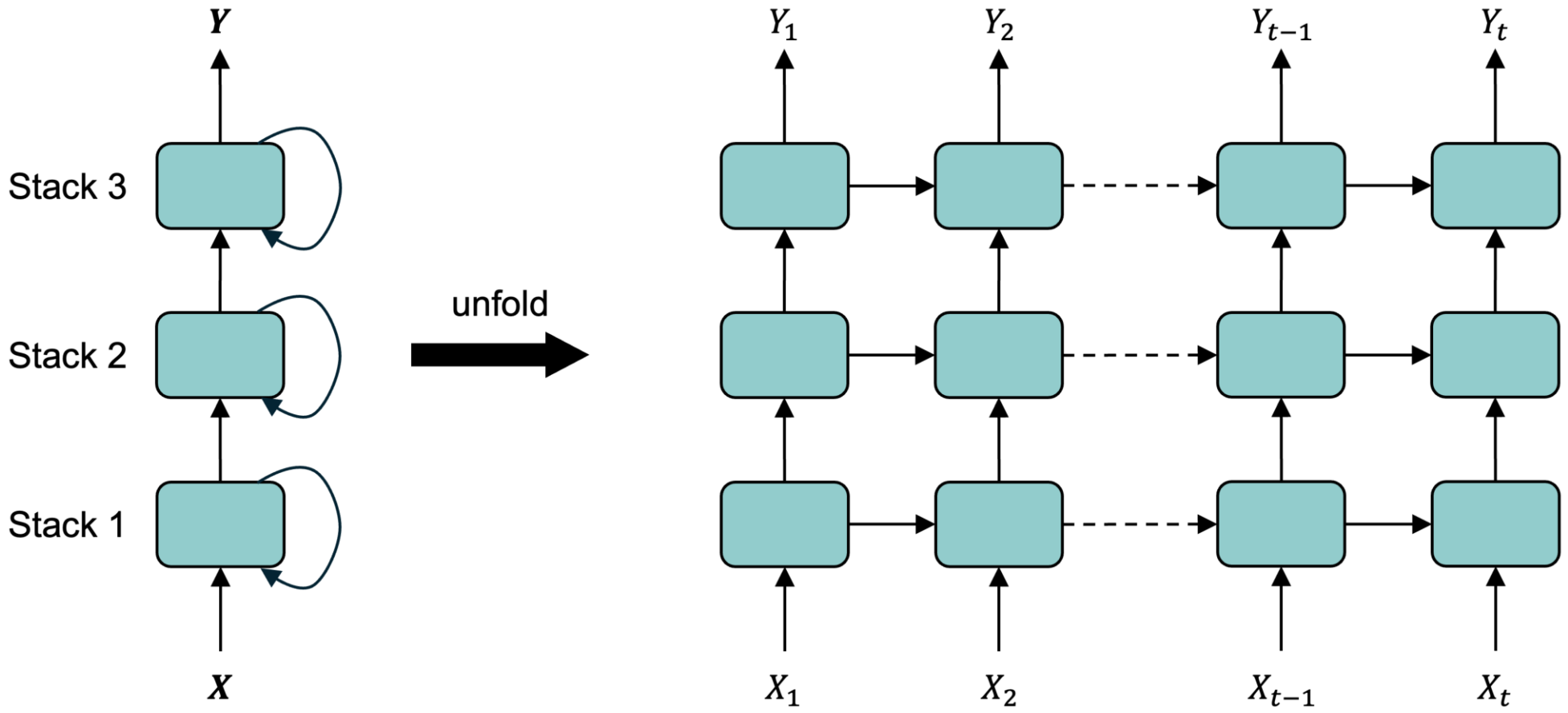
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Diagram of a GRU cell.

Source: Christopher Olah (2015), Understanding LSTM Networks, Colah's Blog.

Recurrent layers can be stacked



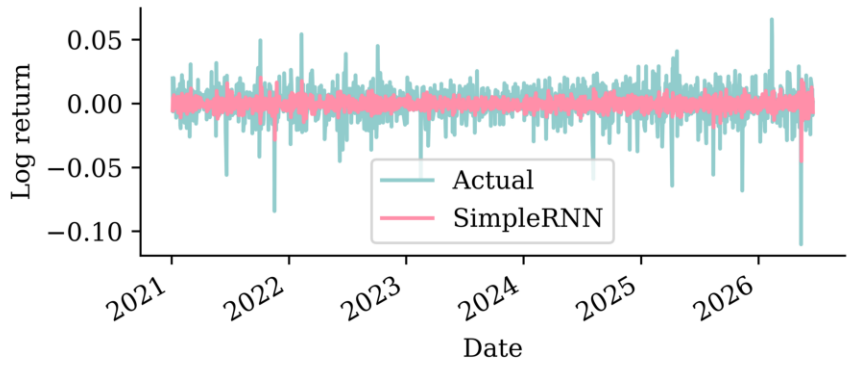
Deep RNN unrolled through time.

Source: Melissa Renard (2025)

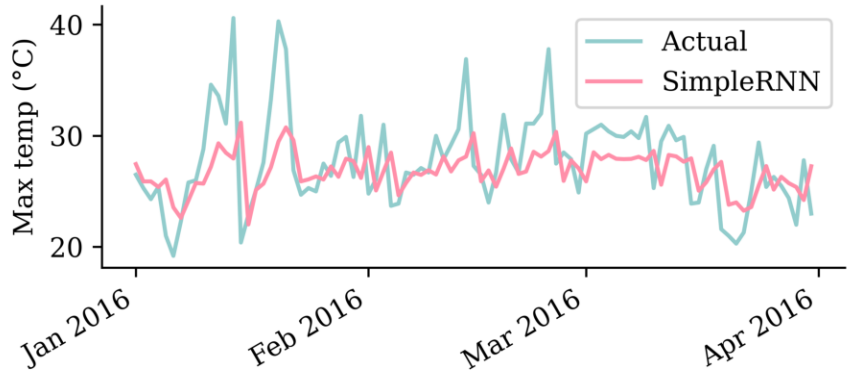
Fitting the SimpleRNN

```
scale = 1 / finance_scale  # for weather: 1/40
model = Sequential([Rescaling(scale), Reshape((-1, 1)), SimpleRNN(64, activation="tanh"), Dense(1)])
model.compile(optimizer="adam", loss="mean_absolute_error")
model.fit(X_train_cba, y_train_cba, validation_data=(X_val_cba, y_val_cba), epochs=500,
    callbacks=[EarlyStopping(patience=15, restore_best_weights=True)], verbose=0)
```

CBA stock price forecasts (test set)



Temperature forecasts (test set)

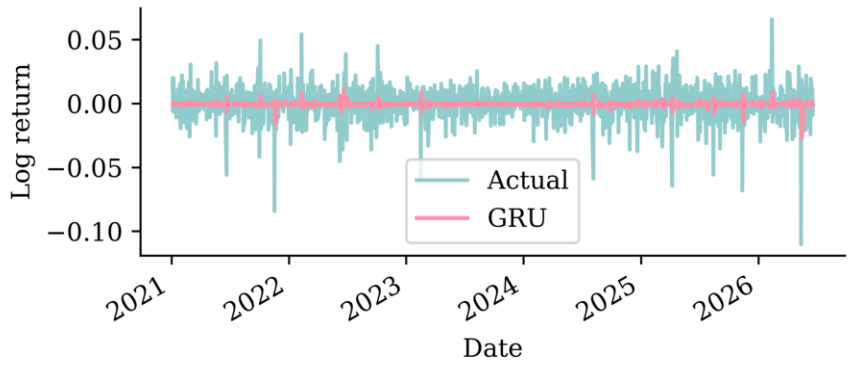


UNSW SYDNEY

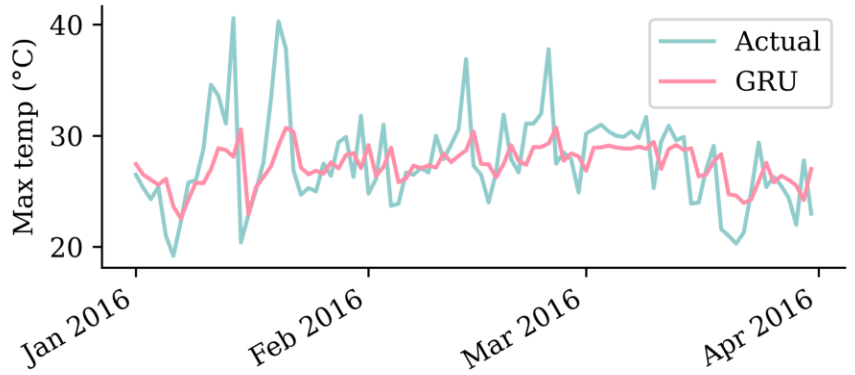
Fitting a GRU

```
scale = 1 / finance_scale  # for weather: 1/40
model = Sequential([Rescaling(scale), Reshape((-1, 1)), GRU(16, activation="tanh"), Dense(1)])
model.compile(optimizer="adam", loss="mean_absolute_error")
model.fit(X_train_cba, y_train_cba, validation_data=(X_val_cba, y_val_cba), epochs=500,
    callbacks=[EarlyStopping(patience=15, restore_best_weights=True)], verbose=0)
```

CBA stock price forecasts (test set)



Temperature forecasts (test set)



Metrics on the validation set

Stock price forecasting

	RMSE ($)
FNN	0.81
SimpleRNN	0.80
Linear	0.77
GRU	0.77
Persistence (1-step)	0.77

Temperature forecasting

	RMSE (°C)
Persistence (1-step)	4.07
Seasonal	3.85
Linear	3.46
SimpleRNN	3.43
FNN	3.40
GRU	3.38

Metrics on the test set

Stock price forecasting

	RMSE ($)
Persistence (1-step)	1.63

Temperature forecasting

	RMSE (°C)
GRU	3.48

Lecture Outline

- Introduction
- Time series data
- Australian financial stocks
- Sydney weather
- Naive baseline forecasts
- Use the recent history
- Simple Recurrent Neural Networks
- Recurrent Neural Networks
- **Multi-step forecasting and multivariate forecasting**
- Car Crash NLP with RNNs

Autoregressive forecasts

Every model so far is *one-step*: it maps the last 40 days to a single next value. To forecast further out, we can feed the model *its own predictions*.

Idea: make the first forecast, then treat it as if it were real and use it to make the next forecast, and so on. For the linear model this reads:

$$\begin{aligned}
\hat{y}_t &= \beta_0 + \beta_1 y_{t-1} + \beta_2 y_{t-2} + \ldots + \beta_n y_{t-n} \\
\hat{y}_{t+1} &= \beta_0 + \beta_1 \hat{y}_t + \beta_2 y_{t-1} + \ldots + \beta_n y_{t-n+1} \\
\hat{y}_{t+2} &= \beta_0 + \beta_1 \hat{y}_{t+1} + \beta_2 \hat{y}_t + \ldots + \beta_n y_{t-n+2} \\
&\vdots \\
\hat{y}_{t+k} &= \beta_0 + \beta_1 \hat{y}_{t+k-1} + \beta_2 \hat{y}_{t+k-2} + \ldots + \beta_n \hat{y}_{t+k-n}
\end{aligned}$$

After n steps the window holds *only* predictions — the model is running entirely on its own output. The same recursion works for *any* one-step model.

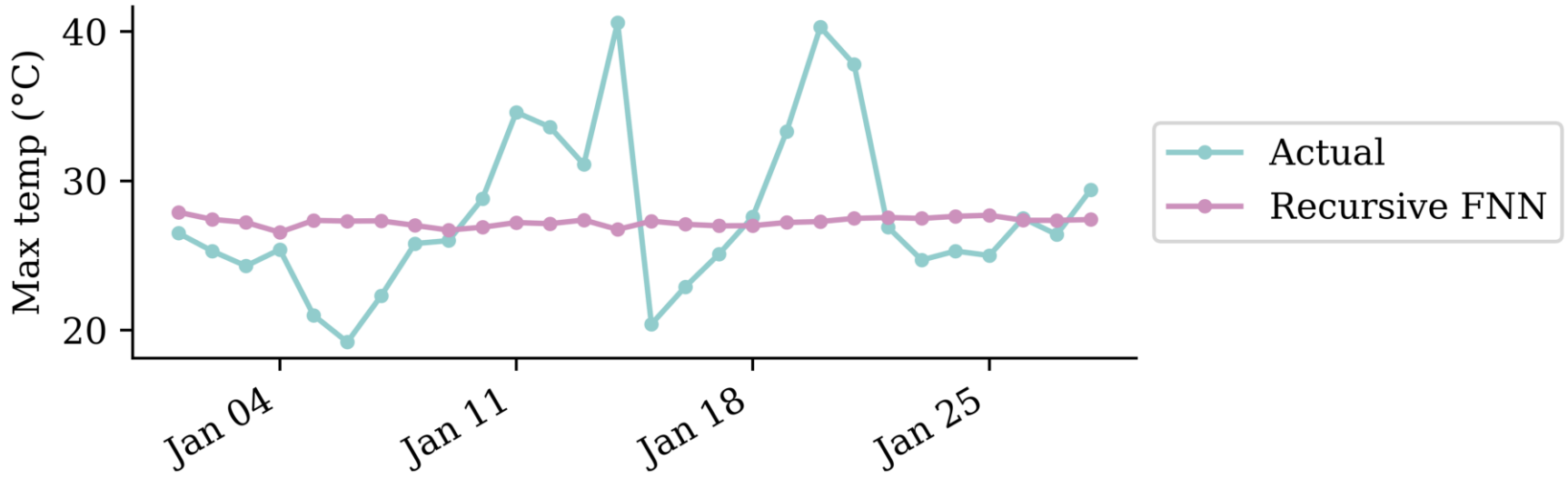
Autoregressive forecasting function

Each step predicts one value, appends it to the window, and drops the oldest input — so the 40-day window slides forward one day at a time.

```python
def autoregressive_forecast(model, window, n_steps):
    """Roll a one-step model forward, feeding each prediction back in."""
    window = np.asarray(window, dtype="float32").copy()
    preds = []
    for _ in range(n_steps):
        next_value = model.predict(window.reshape(1, -1), verbose=0).flatten()[0]
        preds.append(next_value)
        window = np.append(window[1:], next_value)  # drop oldest, add prediction
    return np.array(preds)
```

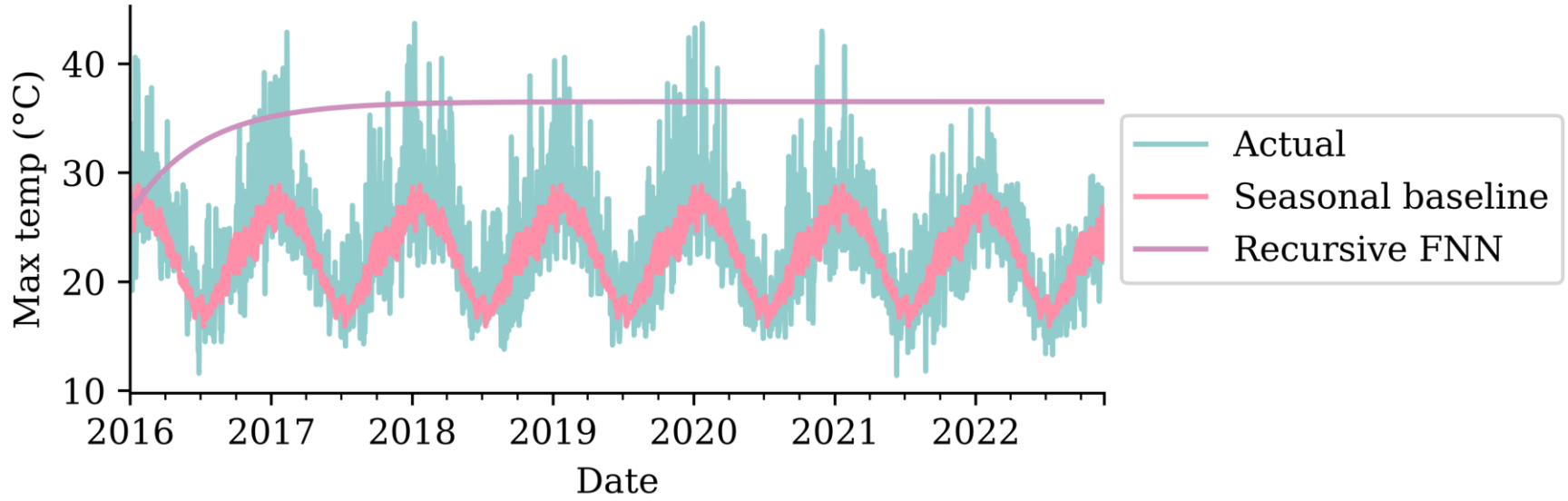
Forecasting a few weeks ahead

Start the model with the last 40 real days, then forecast the next four weeks. Over a short horizon the errors may be small enough.



Forecasting a year ahead

Keep unrolling for the *whole* test period and the small errors compound. The forecast drifts off, loses the seasonal cycle, and settles on a flat plateau, much worse than the trivial baseline.



Multivariate forecast model

Every model so far ended in `Dense(1)` — it predicted a *single* number/time series.

Often we want to forecast *several related series at once* — e.g. the big-four banks, or house-price indices across multiple cities.

An RNN does this with almost no change: feed it a *multivariate* sequence and *widen the output head* so it predicts a vector.

One series out — scalar target.

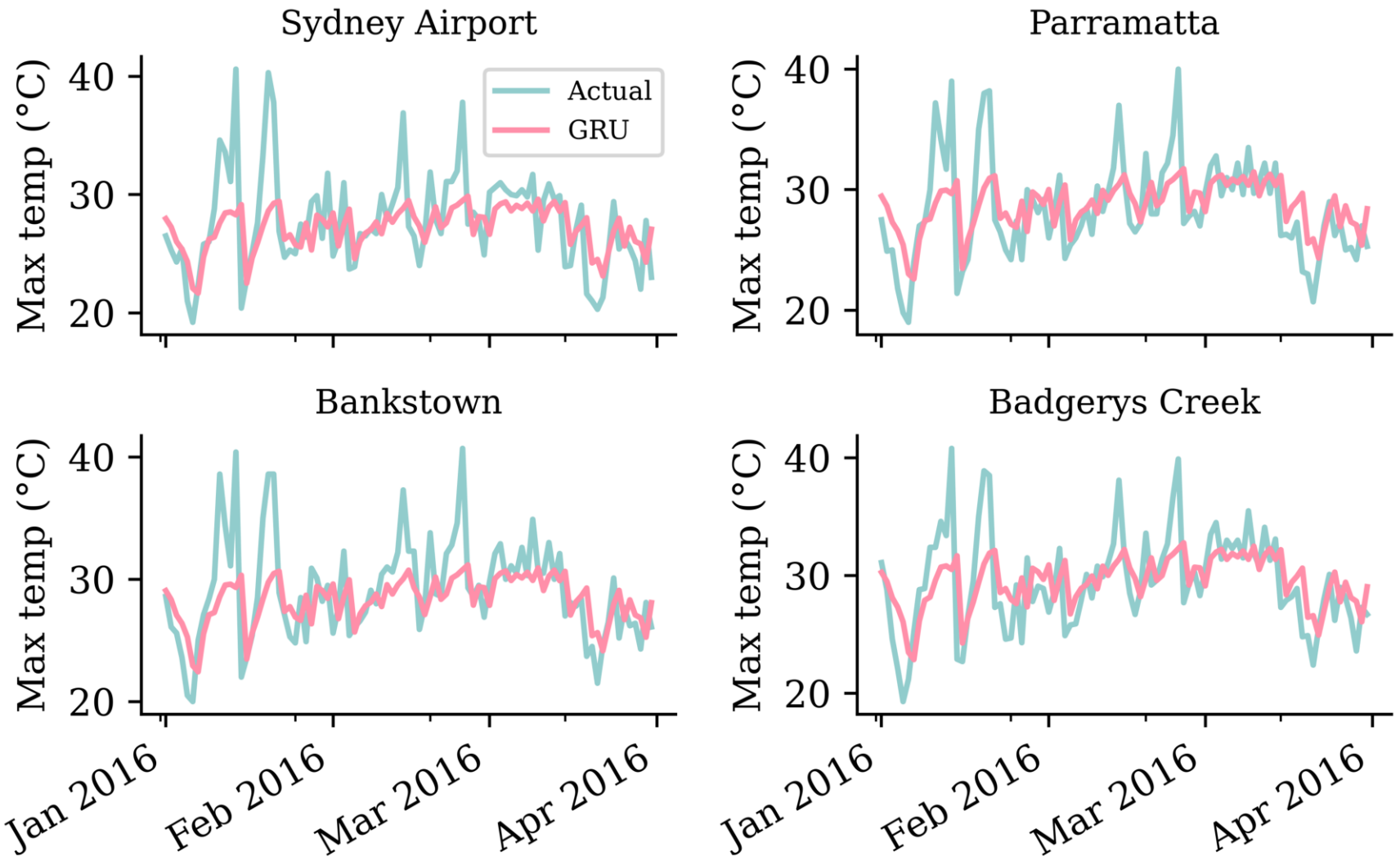
```
model = Sequential([
    Input((seq_len, num_series)),
    GRU(16),
    Dense(1),
])
```

Many series out — vector target.

```
model = Sequential([
    Input((seq_len, num_series)),
    GRU(16),
    Dense(num_series),
])
```

- The input shape `(seq_len, num_series)` is the same — the RNN reads all the series at each step.
- Only the final `Dense` layer changes from `1` to `num_series`, and the loss is averaged over all outputs — so the model can exploit *correlations across series*.

A multivariate example



Lecture Outline

- Introduction
- Time series data
- Australian financial stocks
- Sydney weather
- Naive baseline forecasts
- Use the recent history
- Simple Recurrent Neural Networks
- Recurrent Neural Networks
- Multi-step forecasting and multivariate forecasting
- **Car Crash NLP with RNNs**

Dataset source: Dr Jürg Schelldorfer's GitHub.

Predict injury severity from crash reports

```
features = df["SUMMARY_EN"]
target = LabelEncoder().fit_transform(df["INJSEVB"])

X_main, X_test, y_main, y_test = train_test_split(features, target, test_size=0.2, random
X_train, X_val, y_train, y_val = train_test_split(X_main, y_main, test_size=0.25, random_
X_train.shape, X_val.shape, X_test.shape
```

```
((4169,), (1390,), (1390,))
```

Keep text as sequence of tokens

```
max_length = 500
max_tokens = 1_000

# Build vocabulary using CountVectorizer
count_vect = CountVectorizer(max_features=max_tokens-2, lowercase=True,
    token_pattern=r"(?u)\b\w+\b")
count_vect.fit(X_train)
vocab = ["", "[UNK]"] + list(count_vect.get_feature_names_out())  # 0 = padding, 1 = unkn
word_to_idx = {word: idx for idx, word in enumerate(vocab)}
```

Note: on the TensorFlow backend, you could just use a `keras.layers.TextVectorization` layer, which builds the vocabulary, tokenises, and pads (using `0` for padding and `1` for the unknown token) automatically.

Tokenise

```python
def texts_to_sequences(texts, word_to_idx, max_length):
    sequences = []
    for text in texts:
        tokens = re.findall(r"(?u)\b\w+\b", text.lower())
        seq = [word_to_idx.get(t, 1) for t in tokens][:max_length]  # unknown words → 1
        seq = seq + [0] * (max_length - len(seq))  # pad with zeros
        sequences.append(seq)
    return np.array(sequences)

X_train_txt = texts_to_sequences(X_train, word_to_idx, max_length)
X_val_txt = texts_to_sequences(X_val, word_to_idx, max_length)
X_test_txt = texts_to_sequences(X_test, word_to_idx, max_length)

print(vocab[:5], vocab[len(vocab)//2:(len(vocab)//2 + 5)], vocab[-5:])
```

```
['', '[UNK]', '0', '1', '10'] ['is', 'it', 'its', 'jeep', 'jersey'] ['year', 'years',
'yellow', 'yield', 'zone']
```

The input data is a sequence of integers

```
1 X_train_txt[0]
```

```
array([880, 249, 619, 469, 870, 319,  97, 620,  78, 963, 469, 870, 568,
       620,  78, 392, 958, 849, 493, 870, 493, 224, 620,  78, 605, 818,
        62, 513, 924, 514, 317, 284, 191, 919, 513, 741, 491, 178, 109,
       320, 969,  62, 513, 924, 514, 317, 284, 191, 919, 513, 741, 235,
       178,  78, 691,   1, 900, 788, 870, 161, 605, 818, 741, 957, 313,
       110, 843, 984,  78, 807, 672, 809, 870, 675, 823, 957,  65, 507,
        49, 587, 870, 900, 382, 957, 604, 110, 874, 968, 601,  94, 135,
       219, 134, 870, 885, 620, 870, 249, 943,  78,  18, 179,   1,   1,
       957, 840, 134, 870, 493, 355, 818, 469, 513, 883, 954, 387, 870,
       788, 889, 193, 815, 501, 244, 919, 521, 944,  78,  25, 297,   1,
       957, 606, 469, 513, 924, 119, 870, 759, 493, 976, 501, 486, 889,
       411, 843, 975, 870, 529, 920, 420, 943, 154, 502, 521, 126, 943,
       231, 870, 919, 502, 306, 761,  78, 667, 949, 984,   1, 531,   1,
       119, 870, 493, 397, 870, 320, 943, 840, 469, 870, 568, 620, 870,
       493, 469,   1, 889, 870, 119, 667, 949, 944, 334, 870, 493, 110,
       848, 870, 398, 620, 943, 984, 502, 398, 521, 239, 166, 950, 180,
```

Feed LSTM a sequence of one-hots

```
random.seed(42)
one_hot_model = Sequential([Input(shape=(max_length,), dtype="int64"),
    # A frozen identity-matrix embedding maps each token id to its one-hot vector,
    # while mask_zero lets the LSTM skip the padding (0) positions.
    Embedding(max_tokens, max_tokens, embeddings_initializer="identity",
        trainable=False, mask_zero=True),
    Bidirectional(LSTM(24)),
    Dense(1, activation="sigmoid")])
one_hot_model.compile(optimizer="adam",
    loss="binary_crossentropy", metrics=["accuracy"])
one_hot_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 500, 1000)	1,000,000
bidirectional (Bidirectional)	(None, 48)	196,800
dense (Dense)	(None, 1)	49

Total params: 1,196,849 (4.57 MB)
Trainable params: 196,849 (768.94 KB)
Non-trainable params: 1,000,000 (3.81 MB)

Fit & evaluate

```
es = keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True,
    monitor="val_accuracy", verbose=2)
one_hot_model.fit(X_train_txt, y_train, epochs=1_000, callbacks=[es],
    validation_data=(X_val_txt, y_val), batch_size=128, verbose=0);
```

```
one_hot_model.evaluate(X_train_txt, y_train, verbose=0, batch_size=1_000)
```

```
[0.2810576558113098, 0.894698977470398]
```

```
one_hot_model.evaluate(X_val_txt, y_val, verbose=0, batch_size=1_000)
```

```
[0.3090786337852478, 0.8992805480957031]
```

Custom embeddings

```
embed_lstm = Sequential([Input(shape=(max_length,), dtype="int64"),
    # mask_zero masks only padding (0); the unknown token (1) keeps its own
    # learnable embedding, so "a rare word appeared here" stays a usable signal.
    Embedding(input_dim=max_tokens, output_dim=32, mask_zero=True),
    Bidirectional(LSTM(24)),
    Dense(1, activation="sigmoid")])
embed_lstm.compile("adam", "binary_crossentropy", metrics=["accuracy"])
embed_lstm.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 500, 32)	32,000
bidirectional_1 (Bidirectional)	(None, 48)	10,944
dense_1 (Dense)	(None, 1)	49

Total params: 42,993 (167.94 KB)
Trainable params: 42,993 (167.94 KB)
Non-trainable params: 0 (0.00 B)

Fit & evaluate

```
es = keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True,
    monitor="val_accuracy", verbose=2)
embed_lstm.fit(X_train_txt, y_train, epochs=1_000, callbacks=[es],
    validation_data=(X_val_txt, y_val), batch_size=128, verbose=0);
```

```
embed_lstm.evaluate(X_train_txt, y_train, verbose=0, batch_size=1_000)
```

```
[0.2320377081632614, 0.91460782289505]
```

```
embed_lstm.evaluate(X_val_txt, y_val, verbose=0, batch_size=1_000)
```

```
[0.2636644244194031, 0.9187050461769104]
```

```
embed_lstm.evaluate(X_test_txt, y_test, verbose=0, batch_size=1_000)
```

```
[0.2922019362449646, 0.9122301936149597]
```

Package Versions

```
1 from watermark import watermark
2 print(watermark(python=True, packages="keras,matplotlib,numpy,pandas,seaborn,scipy,torch"
```

```
Python implementation: CPython
Python version       : 3.14.5
IPython version      : 9.15.0

keras     : 3.15.0
matplotlib: 3.11.0
numpy     : 2.5.0
pandas    : 3.0.3
seaborn   : 0.13.2
scipy     : 1.18.0
torch     : 2.12.1
```

Glossary

- autoregressive forecasting
- forecasting
- GRU
- LSTM
- one-step/multi-step ahead forecasting
- persistence forecast
- recurrent neural networks
- SimpleRNN

References

Benidis, K., Rangapuram, S. S., Flunkert, V., Wang, Y., Maddix, D., Turkmen, C., Gasthaus, J., Bohlke-Schneider, M., Salinas, D., Stella, L., et al. (2022). Deep learning for time series forecasting: Tutorial and literature survey. *ACM Computing Surveys*, *55*(6), 1–36.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.

Richman, R., & Wuthrich, M. V. (2019). Lee and Carter go machine learning: Recurrent neural networks. *Available at SSRN 3441030*.